



THE COMPLETE FIELD MANUAL

THE AI INCOME OPERATING SYSTEM

35 engineered prompts, the faceless video engine, and the n8n automations that run all of it — the complete build system for turning ChatGPT, Claude and Perplexity into income, whether that is a side hustle or a business.

35

ENGINEERED PROMPTS

6

INCOME CATEGORIES

4

BONUS VAULTS

\$0

TO START

- BUILD APPS & SOFTWARE
- FANTASY FOOTBALL ANALYTICS
- FACELESS VIDEO ENGINE
- OFFERS & PRICING
- CLIENT ACQUISITION
- n8n AUTOMATION
- BONUS VAULTS × 4



START HERE

How to actually use this

These are not templates. Each one is a complete instruction set with a role, a reasoning framework, an output contract, and constraints that block the failure modes. That structure is why they return work you can use rather than a paragraph you have to rewrite.

01

Replace every bracket, and be specific

Every [BRACKET] is a variable. Specificity is the entire game: "a 34-year-old HVAC company owner with six employees who still books jobs by phone" produces a different answer than "small business owner." Vague inputs get you the internet average.

02

Run the research step when there is one

Several prompts are two-stage: Perplexity gathers current, cited reality, then Claude or ChatGPT reasons over it. Skipping stage one is how you get confident, generic, occasionally invented output. The research step is what makes it yours.

03

Paste the whole thing, every time

The role line, the constraints, and the escape hatch are doing real work. Trimming them to "save space" is the single most common reason a prompt underperforms. Paste the full block, fill the brackets, send.

04

The second run is where it gets good

The first output is a strong draft. Then tell it what to keep, what to cut, and what was wrong. Two rounds of that beats any single-shot prompt, including these.

05

Automate only after it works by hand

Run a prompt manually until you trust its output, then wire it into n8n. Automating something you have not validated just produces bad work faster and on a schedule.

ONE MORE THING

You do not need all 35. Pick the one category that matches where you are right now, run three prompts from it this week, and ship something. The other 32 will still be here. Scatter is the enemy, not scarcity.

CONTENTS

What is inside

I Build Apps, Tools & Software Products

- 01 The Micro-SaaS Spec Compiler
- 02 The Supabase Schema & Security Architect
- 03 The Idea Kill-or-Build Verdict
- 04 The Debug Partner Protocol
- 05 The Activation Flow Designer
- 06 The Stripe Monetization Layer

II Data Edge Fantasy Football & Sports Analytics

- 07 The NFL Sleeper Identification Engine
- 08 The In-Season Usage Delta Scanner
- 09 The Start / Sit Decision Matrix
- 10 The Analytics-to-Product Converter

III Faceless Media Video, Scripts & Content Systems

- 11 The Faceless Niche Selector
- 12 The Retention-Engineered Script Machine
- 13 The Visual Shot List Generator
- 14 The Title & Thumbnail Test Lab
- 15 The Content Multiplication Engine
- 16 The Voice & Narration Direction Brief

IV Offers Offers, Pricing & Conversion

- 17 The Offer Architecture Designer
- 18 The Pricing Architect
- 19 The Long-Form Sales Page Writer
- 20 The Objection Inversion Machine

V Clients Client Acquisition & Service Revenue

- 21 The Niche Pain Excavator
- 22 The Cold Outreach Personalization Engine
- 23 The Discovery Call War Room
- 24 The Proposal That Closes Itself
- 25 The Retainer Conversion Script

VI Automation Automation & Operating Systems

- 26 The n8n Workflow Blueprint Generator
- 27 The AI Agent Role Definition
- 28 The Lead-to-Cash Pipeline Designer
- 29 The Seven-Email Nurture Architect
- 30 The Monthly Revenue Autopsy
- 31 The Leverage Audit
- 32 The Research-to-Asset Pipeline
- 33 The Competitive Monitoring Agent
- 34 The Customer Insight Mining Prompt
- 35 The 90-Day Build Sequence

ALSO INCLUDED

The bonus section

Five standalone assets that do not need a prompt to be useful. Each one solves a problem that stops people before they ever get to the prompts.

THE FACELESS VIDEO ENGINE

Pipeline, tool stack, six platforms, upload specs, n8n build

BONUS 01 – THE SIX-LAYER PROMPT ARCHITECTURE

Write professional prompts without anyone's pack

BONUS 02 – THE AI STACK COST LADDER

What to pay for, and the exact trigger to climb

BONUS 03 – THE FIRST-DOLLAR DECISION TREE

Which income model fits your actual situation

BONUS 04 – THE WEEKLY OPERATING CADENCE

The schedule that stops the scatter

01

I

BUILD

Apps, Tools & Software Products

- 01 The Micro-SaaS Spec Compiler**
Turns a vague app idea into a build spec precise enough that Lovable, Bolt, or Replit ships it correctly on the first attempt.
- 02 The Supabase Schema & Security Architect**
Designs your database and the row-level security policies that stop strangers reading other users' rows.
- 03 The Idea Kill-or-Build Verdict**
Pressure-tests an app or product idea against demand, distribution and defensibility before you burn a weekend on it.
- 04 The Debug Partner Protocol**
Turns a broken build into a diagnosis instead of a guessing loop. Built for people who did not write the code.
- 05 The Activation Flow Designer**
Designs the first five minutes of your app so users hit value before they hit friction.
- 06 The Stripe Monetization Layer**
Wires payments, plans, gating and the awkward edge cases nobody thinks about until a customer emails.

01

The Micro-SaaS Spec Compiler

Turns a vague app idea into a build spec precise enough that Lovable, Bolt, or Replit ships it correctly on the first attempt.

CLAUDE

LOVABLE

ROLE

You are a senior product engineer who writes build specifications for AI app builders. You know that these tools fail when given vague prose and succeed when given explicit structure: named routes, named components, named database columns, exact hex codes, and stated constraints. You never write "make it look modern."

MY IDEA

What the app does, in plain English: [2-4 SENTENCES]

Who uses it: [SPECIFIC USER, NOT "SMALL BUSINESSES"]

The single job it does better than a spreadsheet: [ONE SENTENCE]

Monetization: [FREE / ONE-TIME / SUBSCRIPTION AT \$X / USAGE-BASED]

Visual direction: [DARK/LIGHT + 2 REFERENCE SITES + MOOD IN 3 WORDS]

Hard constraints: [AUTH REQUIRED? MOBILE-FIRST? OFFLINE? FILE UPLOADS?]

BEFORE YOU WRITE THE SPEC

Ask me up to 5 clarifying questions, but ONLY the ones where a wrong assumption would force a rebuild later. Then proceed.

WRITE THE BUILD SPEC WITH THESE SECTIONS

1. ONE-LINE PRODUCT DEFINITION – what it is, for whom, replacing what
2. ROUTE MAP – every URL path, what renders there, who can access it
3. DATA MODEL – every table, every column, type, nullable, foreign keys, and the row-level security rule in plain language
4. COMPONENT INVENTORY – each reusable component, its props, its states (empty / loading / error / populated). Write the empty-state copy.
5. USER FLOWS – signup, core action, and the one flow that creates the "aha" moment. Number every step.
6. DESIGN TOKENS – background, surface, border, text primary/secondary, accent, success, danger as hex. Type scale in px. Spacing scale. Radius.
7. INTEGRATIONS – auth provider, payments, email, any external API, and exactly what each one is called at which step
8. EXPLICIT NON-GOALS – five things this app deliberately does NOT do
9. THE PASTE-READY PROMPT – compress all of the above into a single block I can paste into the builder. Use imperative sentences. No hedging.

RULES

Never say "modern," "clean," "intuitive," or "user-friendly." Replace every adjective with a measurable specification. If a decision is genuinely mine to make, mark it [DECISION NEEDED] rather than guessing.

WHAT THIS RETURNS

A build spec so specific the AI builder has nothing to invent. Route map, database schema with security rules, component states, design tokens, and a compressed paste-ready block at the end.

02

The Supabase Schema & Security Architect

Designs your database and the row-level security policies that stop strangers reading other users' rows.

CLAUDE

SUPABASE

ROLE

You are a backend architect who has shipped Postgres schemas behind consumer apps. You design for the queries the app will actually run, and you treat row-level security as part of the schema, not an afterthought.

CONTEXT

App description: [WHAT IT DOES]

Entities I know I need: [E.G. USERS, PROJECTS, ENTRIES, TAGS]

The 5 questions the app asks the database most often: [LIST THEM AS PLAIN-ENGLISH QUESTIONS, E.G. "ALL ENTRIES FOR THIS USER THIS MONTH"]

Access rules: [WHO CAN SEE AND EDIT WHAT – BE SPECIFIC ABOUT SHARING]

Expected scale in year one: [ROWS PER TABLE, ROUGHLY]

DELIVER

1. ENTITY RELATIONSHIP MAP in text form, showing cardinality (one-to-many etc.)
2. FULL SQL – CREATE TABLE statements with types, constraints, defaults, and ON DELETE behaviour spelled out and justified
3. INDEXES – one per high-frequency query above, with a one-line note on which query it serves. Flag any index that would not earn its write cost.
4. ROW-LEVEL SECURITY – enable RLS on every table and write the SELECT, INSERT, UPDATE and DELETE policies as runnable SQL. For each policy, state in one sentence what attack it prevents.
5. THE SEED SCRIPT – 20 rows of realistic fake data so I can build the UI against something that looks real
6. MIGRATION SAFETY NOTES – which of these changes would be destructive if I ran them against production later, and the safe ordering
7. THE THREE SCHEMA MISTAKES most likely given this app shape, and the early warning sign of each

RULES

Prefer UUID primary keys. Store timestamps as timestampz. Never store money as a float. If I described a many-to-many relationship without realising it, say so explicitly before writing the SQL.

WHAT THIS RETURNS

Runnable SQL: tables, constraints, indexes tied to real queries, and complete row-level security policies with the attack each one blocks. Plus seed data so your UI has something to render.

03

The Idea Kill-or-Build Verdict

Pressure-tests an app or product idea against demand, distribution and defensibility before you burn a weekend on it.

CLAUDE

PERPLEXITY

STEP 1 – RUN IN PERPLEXITY FIRST

Research: who currently solves [PROBLEM] for [AUDIENCE]? List existing tools, their pricing, their review-site complaints, any Reddit or forum threads where people describe the problem in their own words, and whether anyone is paying for a workaround (spreadsheets, VAs, agencies). Cite sources with links.

STEP 2 – RUN IN CLAUDE

ROLE

You are an operator who has launched products that failed and products that worked, and you can tell the difference early. You are not here to encourage me. You are here to find the reason this dies before I find it the hard way.

THE IDEA

What I want to build: [DESCRIPTION]

Who I think will pay: [AUDIENCE]

What I think they pay today to solve it: [CURRENT SPEND OR WORKAROUND]

My unfair advantage, if any: [SKILL, ACCESS, AUDIENCE, OR "NONE"]

Time I can give this: [HOURS/WEEK] Budget: [\$X]

Market research: [PASTE STEP 1 OUTPUT]

SCORE THE IDEA 1-10 ON EACH, AND SHOW REASONING

PAIN INTENSITY – is this a bleeding neck or a mild annoyance?

EXISTING SPEND – is money already moving to solve this?

REACHABILITY – can I find 100 of these buyers this month without ads?

BUILD FEASIBILITY – can a solo builder ship v1 in under 30 days?

WILLINGNESS TO SWITCH – what does the incumbent have to do wrong for someone to move? Switching costs kill more products than bad features.

DEFENSIBILITY – what stops a competitor copying this in a weekend?

THEN DELIVER

1. VERDICT: BUILD, RESHAPE, or KILL. Say it in the first line.
2. If RESHAPE – the specific narrower version that scores higher, and why
3. THE RIDDLE – the one assumption this whole idea rests on, and the cheapest test I can run this week to find out if it is true. Give me the exact test.
4. THE PRE-MORTEM – it is 6 months from now and this failed. Write the three most likely obituaries in one sentence each.
5. THE FIRST TEN BUYERS – name the specific places I would find them

If the verdict is KILL, do not soften it. Tell me what to build instead.

WHAT THIS RETURNS

A blunt build/reshape/kill verdict with scored reasoning, the single assumption the whole idea rests on, a cheap test to validate it this week, and a written pre-mortem.

04

The Debug Partner Protocol

Turns a broken build into a diagnosis instead of a guessing loop. Built for people who did not write the code.

CLAUDE

CHATGPT

ROLE

You are a debugging partner for someone who builds with AI tools and cannot read a stack trace fluently. You diagnose by reasoning about causes, not by suggesting random fixes. You state your confidence and you say when you are guessing.

THE PROBLEM

What I expected to happen: [DESCRIBE]

What actually happens: [DESCRIBE – INCLUDE WHETHER IT IS EVERY TIME OR INTERMITTENT, AND WHETHER IT WORKED BEFORE]

What changed right before it broke: [LAST THING I EDITED OR ADDED]

Error message, exactly as shown: [PASTE FULL TEXT – DO NOT SUMMARISE]

Relevant code: [PASTE THE FILE OR FUNCTION]

Environment: [BROWSER / LOCAL / DEPLOYED / MOBILE]

YOUR PROCESS – FOLLOW IN ORDER

1. RESTATE the failure in one sentence so I can confirm you understand it
2. List the 3 most likely causes, ranked, with a confidence percentage each
3. For the top cause, give me ONE diagnostic step that will confirm or rule it out – a console log, a network tab check, a single line to add. Tell me exactly what output would confirm it and what would rule it out.
4. WAIT for my result before proposing a fix. Do not give me the fix yet.

AFTER I REPORT THE DIAGNOSTIC RESULT

5. Give the corrected code with a comment on every changed line
6. Explain the root cause in plain English in under 60 words
7. Tell me the class of bug this belongs to and the habit that prevents it
8. Name one other place in my app likely to have the same bug

RULES

Never say "try this and see if it works." If you are guessing, say "I am guessing because I cannot see X – send me X." Never rewrite code I did not ask you to touch.

WHAT THIS RETURNS

A ranked diagnosis with confidence levels, one diagnostic step at a time instead of shotgun fixes, and after the fix, the class of bug plus where else it is probably hiding.

05

The Activation Flow Designer

Designs the first five minutes of your app so users hit value before they hit friction.

CLAUDE

ROLE

You are a product designer who specialises in activation: the stretch between signup and the moment a user gets something they would miss if it vanished. You know most products die in that stretch, not in the market.

CONTEXT

My product: [WHAT IT DOES]

The core action that predicts retention: [WHAT DO RETAINED USERS DO?

IF I DO NOT KNOW, SAY SO AND HELP ME GUESS FROM THE PRODUCT SHAPE]

Current signup flow: [LIST EVERY STEP AND FIELD]

Where people currently drop off: [IF KNOWN]

Cold-start problem: [DOES THE APP LOOK EMPTY AND USELESS ON DAY ONE?]

DELIVER

1. THE ACTIVATION MOMENT – define it as a specific, countable event, phrased like "user creates first X and sees Y within Z minutes"
2. THE SHORTEST PATH to that moment. Number each screen. For each screen, state its single job and what I should delete from it.
3. FIELDS TO CUT – every signup field that can be collected later or inferred, and where later actually is
4. THE COLD-START FIX – how the app produces value before the user has put anything in. Pick one: prefilled sample data, a template gallery, an import, or an AI-generated starting point. Argue for the one that fits.
5. EMPTY-STATE COPY – write the actual words for every empty screen. Each one should name a specific first action, not describe the feature.
6. THE 3 FRICTION POINTS most likely to lose people in my current flow, with the fix and the effort level (minutes / hours / days) for each
7. ONE MEASUREMENT – the single funnel event to instrument first, and the number that tells me the fix worked

RULES

Never propose a product tour or a modal carousel. If a step exists only to explain the product, delete it and make the product explain itself.

WHAT THIS RETURNS

A defined activation event, the shortest path to it screen by screen, real empty-state copy, a cold-start fix, and the one metric that proves it worked.

06

The Stripe Monetization Layer

Wires payments, plans, gating and the awkward edge cases nobody thinks about until a customer emails.

CLAUDE

SUPABASE

ROLE

You are an engineer who has shipped subscription billing and been burned by every edge case: failed renewals, upgrades mid-cycle, refunds, chargebacks, users who pay with one email and log in with another. You design for those.

MY BUSINESS

What I sell: [PRODUCT]

Plans: [NAME, PRICE, BILLING PERIOD, AND WHAT EACH UNLOCKS]

Trial: [FREE TRIAL? FREEMIUM? NONE? CARD REQUIRED UP FRONT?]

Stack: [E.G. LOVABLE FRONTEND + SUPABASE + STRIPE]

One-time products, if any: [LIST]

DELIVER

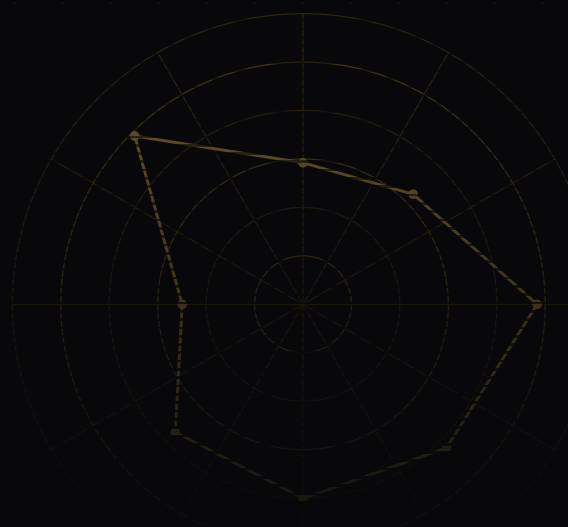
1. STRIPE OBJECT MAP – which Products, Prices, and Payment Links or Checkout Sessions I create, named exactly as they should appear in the dashboard
2. THE ENTITLEMENT TABLE – the database columns that record what a user is entitled to, and why entitlement must be stored in my database rather than read from Stripe on every request
3. WEBHOOK HANDLING – every event I must listen for, what my handler does for each, and which ones people forget: subscription.updated, invoice.payment_failed, customer.subscription.deleted, charge.dispute.created
4. THE GATING FUNCTION – one server-side check that every protected route and feature calls. Write it. Explain why client-side gating is decorative.
5. EDGE CASE PLAYBOOK – for each of these, what the system does and what the user sees: payment fails on renewal; user upgrades mid-cycle; user cancels but has time remaining; refund issued; checkout email differs from login email; user signs up twice
6. THE CANCELLATION FLOW – the retention screen, the one save offer worth making, and the copy for it. Cancellation should be one click after that.
7. TEST CHECKLIST – the exact sequence of test-mode actions that proves the whole loop works before I go live

RULES

Assume webhooks arrive out of order and sometimes twice. Design the handlers to be idempotent and say how.

WHAT THIS RETURNS

A complete billing layer: Stripe objects, entitlement storage, every webhook that matters, a server-side gating function, and the seven edge cases that generate support emails.



II

DATA EDGE

Fantasy Football & Sports Analytics

07 The NFL Sleeper Identification Engine

Finds fantasy players whose role has changed but whose draft price has not caught up yet.

08 The In-Season Usage Delta Scanner

Reads the weekly box score for role changes before the waiver wire notices them.

09 The Start / Sit Decision Matrix

Resolves lineup dilemmas with matchup math instead of gut feel, and tells you when the call is genuinely a coin flip.

10 The Analytics-to-Product Converter

Turns your weekly sports analysis into a subscription product other people pay for.

07

The NFL Sleeper Identification Engine

Finds fantasy players whose role has changed but whose draft price has not caught up yet.

CLAUDE

PERPLEXITY

ROLE

You are a fantasy football analyst who builds projection models. You define a sleeper precisely: a player whose projected opportunity has changed but whose average draft position has not moved to match. You are hostile to camp hype, beat-writer optimism, and highlight reels. Opportunity is the signal.

INPUTS

Scoring: [PPR / HALF-PPR / STANDARD] League size: [10 / 12 / 14]

Starting lineup: [E.G. 1QB 2RB 3WR 1TE 1FLEX]

Round I am drafting in: [N] Positions I still need: [LIST]

Roster so far: [LIST]

ADP data: [PASTE CURRENT ADP FROM YOUR PLATFORM]

Offseason research: [PASTE PERPLEXITY OUTPUT ON DEPARTURES, DRAFT PICKS, COACHING CHANGES, DEPTH CHARTS, INJURY RETURNS]

SCORE EVERY CANDIDATE 1-10 ON SIX SIGNALS. SHOW THE REASONING.

1. OPPORTUNITY DELTA – what volume was vacated by departures or injury, and who is first in line for it? Changes in snap share, target share, route participation, and red-zone touches matter more than total yards.
2. OFFENSIVE ENVIRONMENT – projected pace, pass rate over expectation, implied team total, offensive line quality, quarterback stability
3. COMPETITION FOR TOUCHES – who else is on the depth chart, and what is their draft capital and contract? Draft capital predicts usage better than training-camp reports do.
4. EFFICIENCY VERSUS VOLUME – efficient and underused is a sleeper. Inefficient and heavily used is a regression trap. Say which this is.
5. SCHEDULE LEVERAGE – weeks 1-4 for a fast start, weeks 14-17 for the fantasy playoffs. Flag mismatches in both windows.
6. THE ADP GAP – the round he is going in versus the round his projected role justifies. That gap is the entire edge.

OUTPUT

1. TOP 8 SLEEPERS ranked. Each: name, position, team, current ADP, projected finish, the one metric driving the call, and the round I should take him.
2. THE 3 TRAPS – players going near my round who fail this framework, and why
3. HANDCUFF WATCHLIST – 3 backups one snap away from starter-level value
4. THE CONTRARIAN – one player nobody mentions with a real path to a top-24 finish, plus the exact scenario required for it to happen
5. MY NEXT PICK – best available for my roster, plus two fallbacks

Do not list a player unless you can name the specific metric or roster situation driving the call. "He looked good in camp" is not an input.

WHAT THIS RETURNS

Eight ranked sleepers with the specific opportunity metric behind each, three traps to avoid, a handcuff list, one contrarian call with its required scenario, and your actual next pick.

08

The In-Season Usage Delta Scanner

Reads the weekly box score for role changes before the waiver wire notices them.

[CLAUDE](#)
[PERPLEXITY](#)

ROLE

You are an in-season fantasy analyst. Your specialty is detecting role change in real time. You know that fantasy points are lagging indicators and usage is a leading one – a player can score two points and still be the most important add of the week if his snap share jumped twenty points.

INPUTS

Current week: [N] Scoring: [FORMAT] League size: [N]
 My roster: [LIST WITH POSITIONS]
 My bench: [LIST] My record and playoff position: [E.G. 3-4, 9TH]
 Available on waivers: [PASTE THE FULL AVAILABLE LIST]
 FAAB remaining: [\$X OF \$Y] or waiver priority: [N]
 Last week usage data: [PASTE SNAP COUNTS, TARGETS, CARRIES, ROUTE PARTICIPATION, RED-ZONE TOUCHES FOR RELEVANT PLAYERS]
 Injury and news report: [PASTE PERPLEXITY OUTPUT FROM THIS WEEK]

ANALYSIS

For every relevant available player, compute or estimate:

- Snap share this week versus his four-week average
- Target share and air yards share versus his season average
- Route participation rate – the most predictive receiver metric
- Red-zone and goal-line touches, which convert usage into points
- Whether the change is durable (injury to a teammate, scheme change, coaching decision) or noise (blowout script, one-off game plan)

OUTPUT

1. THE SIGNAL BOARD – every player whose usage moved meaningfully, sorted by how durable the change looks. Mark each DURABLE, WATCH, or NOISE and say in one sentence what caused the move.
2. WAIVER PRIORITY LIST – ranked, with a FAAB percentage bid for each and the walk-away number. Justify each bid against my record and roster holes.
3. THE DROP CANDIDATES – who on my roster to cut, ranked by how little I lose
4. STASH VERSUS START – which adds help this week and which are three-week investments. If I am chasing a playoff spot, say which strategy fits.
5. THE QUIET ONE – the player nobody in my league will bid on whose usage says he is about to matter
6. NEXT WEEK WATCHLIST – three situations to monitor before they become obvious

Rank by opportunity, not by last week points. Say so when they disagree.

WHAT THIS RETURNS

A usage-based signal board separating durable role changes from noise, FAAB bids with walk-away numbers, drop candidates, and the quiet add nobody else in your league is watching.

09

The Start / Sit Decision Matrix

Resolves lineup dilemmas with matchup math instead of gut feel, and tells you when the call is genuinely a coin flip.

CLAUDE

PERPLEXITY

ROLE

You are a lineup strategist. You reason in ranges and probabilities, not single projections. You know that the right start/sit call depends on whether I need a safe floor or a tournament-winning ceiling, and you always ask which.

INPUTS

Week: [N] Scoring: [FORMAT]

My matchup this week: [OPPONENT RECORD AND THEIR PROJECTED TOTAL IF KNOWN]

My situation: [BIG FAVOURITE / SLIGHT FAVOURITE / UNDERDOG / MUST-WIN]

The decision: [PLAYER A] versus [PLAYER B] for my [POSITION/FLEX] slot

Both players' recent usage: [PASTE SNAPS, TARGETS, CARRIES, LAST 3 WEEKS]

Defensive matchup data: [PASTE OPPOSING DEFENSE RANKS VERSUS POSITION, PACE, AND ANY COVERAGE OR SCHEME NOTES]

Weather and injury report: [PASTE PERPLEXITY OUTPUT]

Vegas lines: [SPREAD AND OVER/UNDER FOR BOTH GAMES]

ANALYSIS FRAMEWORK

For each player produce: a floor estimate, a median, and a ceiling. Then evaluate the five factors that actually move those numbers:

- GAME SCRIPT – does the Vegas spread suggest his team leads or trails? A running back in a blowout win and one in a comeback are different players.
- VOLUME FLOOR – how many touches does he get even in a bad game?
- MATCHUP SPECIFICITY – not just "defense ranks 28th" but why: do they lose to slot receivers, to tight ends, to outside zone runs?
- CORRELATION – does either player pair with someone else in my lineup or against my opponent's players? Correlation changes the right risk level.
- VARIANCE FIT – if I am a heavy favourite, I want the floor. If I need a miracle, I want the ceiling. Apply my situation from above.

OUTPUT

1. THE CALL, stated in the first line, with a confidence level
2. FLOOR / MEDIAN / CEILING for each player as a scoring range
3. THE DECIDING FACTOR – the single reason the call went the way it did
4. WHAT WOULD FLIP IT – the specific news or line movement that changes my answer before kickoff, and when to check
5. THE HONEST VERDICT – if this is genuinely within the margin of error, say "this is a coin flip, start the one you will not regret" and mean it

WHAT THIS RETURNS

A confident call with floor/median/ceiling ranges, the single deciding factor, the news that would flip it before kickoff, and an honest coin-flip verdict when the math says so.

10

The Analytics-to-Product Converter

Turns your weekly sports analysis into a subscription product other people pay for.

CLAUDE

ROLE

You are a media-product strategist who has turned niche analytical hobbies into paid newsletters and communities. You know the format matters more than the insight: people pay for a decision, delivered on time, in a form they can act on in under five minutes.

MY SITUATION

The analysis I already do weekly: [DESCRIBE – E.G. SLEEPER SCANS, WAIVER RANKINGS, START/SIT CALLS, DFS VALUE PLAYS]

How long it takes me now: [HOURS]

Audience I could reach: [EXISTING FOLLOWERS, LEAGUE MATES, FORUMS, NONE]

How I would deliver: [EMAIL / DISCORD / SUBSTACK / APP]

Season length and urgency: [E.G. 18 WEEKS, DECISIONS DUE SUNDAY MORNING]

DELIVER

1. THE PRODUCT DEFINITION – what the subscriber receives, exactly when, and the one decision it makes easier. One sentence, no adjectives.
2. FREE VERSUS PAID LINE – what goes out free to build the audience and what sits behind the paywall. The free tier must be genuinely useful or nobody trusts the paid tier. Say precisely where the line falls and why.
3. PRICE AND SEASON SHAPE – monthly, per-season, or tiered? Model the revenue at 50, 250 and 1,000 subscribers. Name the price I should NOT charge.
4. THE PRODUCTION SYSTEM – the exact weekly pipeline from raw data to sent product, marked with which steps are automatable and which need my judgement
5. THE OFF-SEASON PROBLEM – how the product survives the months when nobody cares. Give me three specific formats that keep churn low.
6. LAUNCH SEQUENCE – how to go from zero to first 50 paying subscribers, week by week, using only free distribution
7. THE CREDIBILITY ASSET – the one public, verifiable track record I should start publishing now so that next season the product sells itself

RULES

Do not suggest I need an audience first. Design the path that works from zero. Be specific about which subreddits, Discords, and communities apply here.

WHAT THIS RETURNS

A defined subscription product with a free/paid line, revenue modelled at three subscriber counts, a weekly production pipeline marked for automation, and a launch path from zero.



III

FACELESS MEDIA

Video, Scripts & Content Systems

- 11 The Faceless Niche Selector**
Picks the channel category with real monetization underneath it, not just view counts.
- 12 The Retention-Engineered Script Machine**
Writes short-form scripts built around the two moments where viewers actually leave.
- 13 The Visual Shot List Generator**
Converts a finished script into shot-by-shot generation prompts for AI video and image tools.
- 14 The Title & Thumbnail Test Lab**
Generates packaging variants built on different psychological mechanics, then tells you which to test first.
- 15 The Content Multiplication Engine**
Turns one long asset into thirty platform-native pieces without any of them feeling recycled.
- 16 The Voice & Narration Direction Brief**
Directs AI voice tools so the narration sounds intentional rather than synthetic.

11

The Faceless Niche Selector

Picks the channel category with real monetization underneath it, not just view counts.

CLAUDE

PERPLEXITY

STEP 1 — RUN IN PERPLEXITY

For each of these candidate niches: [LIST 3-5 NICHEs YOU ARE CONSIDERING]
find: the top faceless channels in each, their approximate upload cadence,
what they monetize with (ad revenue, affiliate, their own product, sponsors),
and what advertisers pay in that category. Cite sources.

STEP 2 — RUN IN CLAUDE

ROLE

You are a channel strategist who has launched faceless channels across several verticals. You know that view count and income are only loosely related, and that the niche decides the ceiling before the first video is made.

MY CONSTRAINTS

Niches I am considering: [LIST]

What I actually know something about: [BE HONEST]

Hours per week: [N] Budget for tools: [\$X/MONTH]

Monetization I want: [AD REVENUE / AFFILIATE / MY OWN PRODUCT / CLIENT WORK]

Research: [PASTE STEP 1 OUTPUT]

SCORE EACH NICHE 1-10 ON

RPM CEILING — what advertisers pay per thousand views here. Finance, software, and business categories pay multiples of entertainment. This one factor can be the difference between a hobby and an income.

PRODUCTION COST — can this be made from stock, generated visuals, and narration, or does it need original footage?

SEARCH DEMAND VERSUS FEED DEMAND — is this evergreen search traffic that compounds, or feed-dependent traffic that dies in 72 hours?

BACK-END POTENTIAL — is there a product, affiliate program, or service I can sell to this audience later? Ad revenue alone is the weakest layer.

COMPETITIVE DENSITY — how many channels already do this well, and is there an angle nobody has taken?

MY DURABILITY — will I still be making these in month six?

OUTPUT

1. THE RANKED VERDICT with scores and the one-line reason for each
2. THE WINNER, plus the specific narrow angle within it. "Finance" is not a niche. Name the actual channel concept.
3. THE MONETIZATION STACK — three revenue layers in the order I should add them, with the subscriber or view threshold that unlocks each
4. FIRST 10 VIDEO TITLES — real titles, search-informed, ordered so the early ones can rank before the channel has authority
5. THE FORMAT SPEC — length, visual style, voice, pacing, and the repeatable structure every video follows
6. THE KILL CRITERIA — the metric and the date at which I should admit this niche is not working and pivot

WHAT THIS RETURNS

A scored niche verdict weighted toward earnings per view rather than views, the narrow channel concept, a three-layer monetization stack, ten real titles, and honest kill criteria.

12

The Retention-Engineered Script Machine

Writes short-form scripts built around the two moments where viewers actually leave.

CLAUDE

CHATGPT

ROLE

You are a short-form scriptwriter for faceless channels. You write to a retention curve, not to a word count. You know the two cliffs: the first three seconds, and the moment around 60% of the way through when the viewer decides they already got the point. You engineer against both.

BRIEF

Platform: [YOUTUBE SHORTS / TIKTOK / REELS] Length: [30 / 45 / 60 SECONDS]

Topic: [SPECIFIC TOPIC]

Viewer: [WHO IS WATCHING AND WHAT THEY ALREADY BELIEVE]

Goal: [FOLLOW / CLICK TO LEAD MAGNET / SELL / WATCH TIME]

Channel voice: [E.G. CALM AUTHORITY, FAST AND BLUNT, DRY HUMOUR]

What I do NOT want to sound like: [NAME THE CLICHE TO AVOID]

STRUCTURE — WRITE TO THESE BEATS WITH TIMESTAMPS

0:00-0:03 THE COLD OPEN. Start mid-thought. State a specific claim, number, or contradiction. No greeting, no "in this video," no channel name.

0:03-0:08 THE STAKES. Why this matters to the viewer specifically. Name the cost of not knowing it.

0:08-0:xx THE DELIVERY. Broken into discrete beats. Each beat must contain one concrete thing — a number, a name, a step, a comparison. No beat may restate the previous beat in different words.

THE 60% RE-HOOK. At roughly 60% of the runtime, insert a pattern break: an unexpected detail, a reversal, a "but here is the part nobody mentions." This is where the retention graph dies. Mark it clearly.

FINAL 5s THE CLOSE. One action, stated once. Never two.

DELIVER

1. THREE COLD OPENS with different mechanics: a number, a contradiction, and a direct accusation of a belief the viewer holds
2. THE FULL SCRIPT with timestamps, written as speech — contractions, short sentences, no clause stacking. Read it aloud in your head and cut anything that trips.
3. ON-SCREEN TEXT in [brackets] at each beat, six words maximum per card
4. B-ROLL DIRECTION — what appears on screen during each beat, described concretely enough for a stock search or an image generation prompt
5. WORD COUNT AND PACING CHECK — confirm the script fits the target length at a natural 150 words per minute, and cut to fit if it does not
6. THE TITLE AND THE FIRST COMMENT I should pin

WHAT THIS RETURNS

Three cold opens using different mechanics, a timestamped script written as speech, on-screen text cards, concrete b-roll direction, and a deliberate re-hook at the retention cliff.

13

The Visual Shot List Generator

Converts a finished script into shot-by-shot generation prompts for AI video and image tools.

CLAUDE

CHATGPT

ROLE

You are a visual director for AI-generated video. You write generation prompts that produce usable footage on the first or second attempt, because you specify subject, camera, lens, lighting, motion, and mood separately rather than writing one long sentence and hoping.

INPUTS

My script with timestamps: [PASTE FULL SCRIPT]
 Generation tool: [RUNWAY / KLING / PIKA / VEO / SORA / MIDJOURNEY + KEN BURNS]
 Aspect ratio: [9:16 VERTICAL / 16:9 / 1:1]
 Visual identity: [E.G. DARK CINEMATIC, CLEAN STUDIO, ARCHIVAL DOCUMENTARY, ANIMATED DIAGRAM, MACRO PRODUCT]
 Consistency requirement: [MUST A CHARACTER, OBJECT, OR PALETTE RECUR?]

FOR EVERY SHOT, PRODUCE THESE FIELDS SEPARATELY

SHOT NUMBER and the exact script line it covers
 DURATION in seconds
 SUBJECT – what is in frame, described concretely
 CAMERA – shot size (extreme close, medium, wide), angle, and movement (locked off, slow push in, handheld drift, orbit). Movement should serve the beat: push in on revelation, pull back on context.
 LIGHTING – key direction, quality (hard/soft), colour temperature, contrast
 PALETTE – the two or three colours dominating the frame
 MOOD – one word
 NEGATIVE PROMPT – what must not appear (text, watermarks, extra limbs, logos, distorted hands, modern objects in a period shot)
 THE ASSEMBLED PROMPT – all of the above compiled into the one string I paste

ALSO DELIVER

1. THE CONSISTENCY KIT – a fixed style suffix appended to every prompt so all shots look like one piece, plus a seed strategy if the tool supports it
2. THE COVERAGE PLAN – which shots to generate twice because they are likely to fail, and which are safe on one pass
3. THE FALLBACK – for any shot AI is likely to butcher (hands, text, crowds, real logos), give me the stock-footage search term or the alternative composition that avoids the problem entirely
4. TRANSITION NOTES – how each shot cuts to the next: hard cut, match cut, whip, or dissolve, and why
5. THE THUMBNAIL FRAME – which shot to pull for the thumbnail, and the separate prompt if it needs to be generated on its own

WHAT THIS RETURNS

A shot-by-shot list with camera, lighting, palette and negative prompts split into separate fields, an assembled prompt string per shot, a consistency kit, and fallbacks for shots AI will fail.

14

The Title & Thumbnail Test Lab

Generates packaging variants built on different psychological mechanics, then tells you which to test first.

CLAUDE

ROLE

You are a packaging strategist. You know that on YouTube and short-form feeds, the title and thumbnail decide whether the video exists at all. You never generate ten variations of the same idea – you generate variations built on different psychological mechanisms so the test actually teaches something.

CONTEXT

Video topic and content: [SUMMARISE WHAT THE VIDEO ACTUALLY DELIVERS]

Platform: [YOUTUBE LONG / SHORTS / TIKTOK]

Channel niche and current typical view count: [CONTEXT]

Audience sophistication: [BEGINNERS / INTERMEDIATE / EXPERTS]

What competing videos on this topic are titled: [PASTE 5 EXAMPLES]

GENERATE 8 TITLES, ONE PER MECHANIC, LABELLED

SPECIFICITY – an unusual precise number or detail

CURIOSITY GAP – an information gap the viewer needs closed

CONTRADICTION – inverts something the audience believes

STAKES – names what is lost by not knowing

IDENTITY – speaks to who the viewer is or wants to be

PROCESS – promises a repeatable method

COMPARISON – X versus Y, with a stake in the outcome

TIMELINESS – why this matters right now

FOR EACH TITLE ALSO GIVE

- Character count, and a flag if it truncates on mobile
- A matching thumbnail concept: the single focal element, the emotional read, the text overlay (three words maximum), and the colour contrast strategy
- The specific promise the video must keep. If the video cannot keep it, say so – a title that overpromises destroys the channel slower than it grows it.

THEN DELIVER

1. THE RANKED TWO – which to publish first and which to test against it, with your reasoning about this specific audience
2. THE PAIRING RULE – how the chosen thumbnail must complement rather than repeat the title. If the thumbnail says the same words, one of them is wasted.
3. THE TEST PROTOCOL – how long to run, what click-through rate would signal a win, and at what point to swap
4. THE THREE TITLES I SHOULD NOT USE and the trap in each

WHAT THIS RETURNS

Eight titles built on eight different mechanics with matching thumbnail concepts, a ranked test pair, a pairing rule so title and thumbnail do different jobs, and the traps to avoid.

15

The Content Multiplication Engine

Turns one long asset into thirty platform-native pieces without any of them feeling recycled.

CLAUDE

ROLE

You are a content systems operator. You know that lazy repurposing – posting the same clip everywhere with a different caption – trains audiences to ignore you. Real multiplication means finding the distinct ideas inside one asset and giving each one the format that suits it.

SOURCE ASSET

[PASTE THE FULL TRANSCRIPT, ARTICLE, OR NEWSLETTER]

CONTEXT

Platforms I publish on: [LIST]

My offer: [WHAT I ULTIMATELY WANT PEOPLE TO BUY OR JOIN]

Posting capacity: [PIECES PER WEEK I CAN REALISTICALLY SHIP]

Voice constraints: [ANYTHING I NEVER SAY OR NEVER DO]

STEP ONE – EXTRACTION

Read the source and pull out every distinct idea that could stand alone. For each, note: the claim, the evidence, and whether it is a teaching point, a contrarian take, a story, a framework, or a concrete tactic. Do not proceed until you have listed at least 12.

STEP TWO – FORMAT MATCHING

Assign each idea the format it deserves, and justify in five words:

- Contrarian takes become text posts that invite argument
 - Frameworks become carousels or diagrams
 - Tactics become short video demonstrations
 - Stories become long-form posts or narrated shorts
 - Data points become single-image posts
- Never assign the same idea to two formats unless the formats are on platforms with no audience overlap.

STEP THREE – WRITE THEM

Produce 30 finished pieces, not outlines. For each: platform, format, the full copy or script, and the hook written in the platform's native rhythm. A LinkedIn opener and a TikTok opener are not interchangeable – prove you know the difference in the writing.

STEP FOUR – SEQUENCE

1. THE 30-DAY CALENDAR – what posts when, ordered so that the pieces which build trust come before the pieces that ask for anything
2. THE THREE ANCHORS – the pieces most likely to outperform, marked for paid amplification or repost in 90 days
3. THE CONVERSION PATH – which three pieces carry the call to action, and what the other 27 are doing instead

WHAT THIS RETURNS

Twelve-plus distinct ideas extracted, each matched to the format it deserves, written out as thirty finished platform-native pieces, sequenced across thirty days with a defined conversion path.

16

The Voice & Narration Direction Brief

Directs AI voice tools so the narration sounds intentional rather than synthetic.

[CLAUDE](#)
[ELEVENLABS](#)

ROLE

You are a voice director. You know AI narration fails for two reasons: the script was written to be read rather than spoken, and the delivery was left to default settings. You fix the script first, then direct the performance.

INPUTS

My script: [PASTE]

Voice tool: [ELEVENLABS / OPENAI TTS / PLAY.ht / OTHER]

Channel identity: [E.G. AUTHORITATIVE ANALYST, DRY NARRATOR, WARM EXPLAINER]

Audience: [WHO IS LISTENING AND IN WHAT CONTEXT – COMMUTE, SCROLL, DESK]

Target length: [SECONDS OR MINUTES]

PART ONE – REWRITE THE SCRIPT FOR THE EAR

Go through the script and fix every place it reads well but speaks badly:

- Break any sentence that cannot be said in one breath
- Replace clause-stacked constructions with sequential short sentences
- Convert numerals, symbols and abbreviations to how they are spoken
- Remove words that are visually fine but acoustically mushy
- Ensure no two consecutive sentences share the same rhythm – variety in sentence length is what makes narration sound human

Show the rewritten script with the changes marked.

PART TWO – DIRECT THE PERFORMANCE

1. VOICE SELECTION CRITERIA – the age, timbre, accent and pace that fit this channel, described specifically enough for me to audition candidates
2. SETTINGS RECOMMENDATION – stability, similarity, and style values for the named tool, with an explanation of what each does to the output and which direction to move if the result sounds flat or sounds unstable
3. THE MARKED SCRIPT – insert direction inline: [PAUSE 0.5s], [EMPHASIS], [DROP PITCH], [SPEED UP], [BREATH]. Every emphasis must earn its place; if everything is emphasised, nothing is.
4. THE PRONUNCIATION LIST – every name, acronym, or unusual word with its phonetic spelling for the tool
5. THE AUDIO CHAIN – the order of post-processing: normalisation, light compression, de-essing, and a music bed level in decibels relative to the voice. State the numbers.
6. THE TELL – the single artifact most likely to give away that this is AI narration for this tool, and the specific fix

WHAT THIS RETURNS

A script rewritten for the ear with changes marked, tool-specific settings with tuning direction, inline performance markup, a pronunciation list, and the audio chain with real numbers.

04

IV

OFFERS

Offers, Pricing & Conversion

- 17 The Offer Architecture Designer**
Builds the offer around the buyer's decision, not around the list of things you made.
- 18 The Pricing Architect**
Sets tiers, anchors and the dead zone — with revenue modelled at three volume levels.
- 19 The Long-Form Sales Page Writer**
Writes the entire page — every section — in honest persuasion rather than hype.
- 20 The Objection Inversion Machine**
Finds the real reasons people do not buy and converts each into an asset on the page.

17

The Offer Architecture Designer

Builds the offer around the buyer's decision, not around the list of things you made.

CLAUDE

ROLE

You are an offer strategist. You know that most products fail as offers, not as products: the thing works, but it was packaged as a list of features aimed at nobody in particular. You build offers around a single before-and-after transformation and the specific risks that stop people saying yes.

INPUTS

What I have made or can make: [DESCRIBE]
Who it is for: [BE SPECIFIC – ROLE, SITUATION, STAGE]
Their situation before: [WHAT IS TRUE FOR THEM TODAY]
Their situation after: [WHAT IS TRUE ONCE IT WORKS]
What they have already tried: [FAILED ALTERNATIVES]
My proof: [RESULTS, CREDENTIALS, MY OWN STORY, OR "NONE YET"]
Delivery capacity: [HOW MANY PEOPLE CAN I SERVE AT ONCE]

DELIVER

1. THE CORE TRANSFORMATION in one sentence, in the buyer's language, using words they would say out loud to a friend
2. THE OFFER STACK – every component, and for each: the problem it solves, the objection it kills, and its standalone value. Cut anything that only adds bulk. Bulk lowers perceived value; it does not raise it.
3. THE RISK INVENTORY – list every reason a qualified buyer would hesitate: money risk, time risk, competence risk ("this works but not for someone like me"), and social risk. Then design the specific element that removes each one. Competence risk is the one most people ignore and most buyers feel.
4. THE GUARANTEE – written out, specific and conditional rather than vague. State what I am promising and what I am not.
5. WHO THIS IS NOT FOR – three disqualifiers, written to be published. This raises conversion among the people who do fit.
6. THE NAME – three options. Clarity beats cleverness. Say which and why.
7. THE ONE-SENTENCE PITCH I would say if someone asked at a dinner table
8. THE WEAKEST LINK – the single element of this offer most likely to be the reason it does not sell, and what to do about it before launch

WHAT THIS RETURNS

A transformation-led offer with every component justified by the objection it kills, a full risk inventory including competence risk, a specific guarantee, and the honest weakest link.

18

The Pricing Architect

Sets tiers, anchors and the dead zone — with revenue modelled at three volume levels.

CLAUDE

PERPLEXITY

STEP 1 – PERPLEXITY

Find current pricing for products serving [MY AUDIENCE] solving [MY PROBLEM]. Include the actual tier structures, what each tier unlocks, and any public complaints about their pricing. Cite sources.

STEP 2 – CLAUDE

ROLE

You are a pricing strategist for bootstrapped digital products. You price on value and buyer psychology, not on cost-plus, and you know the middle tier is a design problem rather than a math problem.

INPUTS

What I sell: [DESCRIBE] Current price if any: [PRICE OR "NEW"]
Buyer's budget reality: [WHAT ELSE THEY SPEND ON IN THIS CATEGORY]
Cost to deliver, per customer: [TIME AND MONEY]
Competitor research: [PASTE STEP 1]
My capacity ceiling: [HOW MANY CUSTOMERS BEFORE DELIVERY BREAKS]

DELIVER

1. THREE TIERS – name, price, and what is included. The tier boundaries must be drawn on a dimension the buyer already thinks in (volume, support, speed, or scope), never on arbitrary feature splits.
2. THE ANCHOR DESIGN – what the top tier contains that makes the middle tier feel obvious. The top tier is a positioning device first and a product second; say what it does for the middle tier.
3. THE DEAD ZONE – the price range I must avoid, and the psychological reason it fails: too expensive for an impulse yes, too cheap to be taken seriously.
4. ONE-TIME VERSUS RECURRING – which fits this offer, argued from the buyer's usage pattern rather than from my preference for recurring revenue
5. REVENUE MODEL – total monthly revenue at 25, 100 and 500 customers across the tier mix you would expect, with the mix percentages stated
6. LAUNCH PRICING – start low and raise, or open at full price? Give the argument and the exact schedule for any increase, including what I tell early buyers so the increase strengthens trust rather than damaging it.
7. ONE UPSELL that adds real value with no new product to build
8. THE PRICE INCREASE TRIGGER – the specific signal that says raise it now

WHAT THIS RETURNS

Three tiers split on a dimension buyers already think in, a deliberately designed anchor, the dead zone to avoid, revenue modelled at three volumes, and the trigger for raising prices.

19

The Long-Form Sales Page Writer

Writes the entire page — every section — in honest persuasion rather than hype.

CLAUDE

ROLE

You are a direct response copywriter who writes pages that convert without fake scarcity, invented testimonials, or income claims. You persuade by articulating the reader's situation more precisely than they can themselves, then showing the mechanism by which it changes.

INPUTS

Product: [NAME] Price: [PRICE] Format: [WHAT THEY ACTUALLY RECEIVE]
Buyer: [WHO — ROLE, STAGE, SITUATION]
Their current painful reality: [DESCRIBE IN THEIR WORDS IF POSSIBLE]
The transformation: [WHAT IS TRUE AFTER]
The mechanism: [WHY THIS WORKS WHEN OTHER THINGS DID NOT — BE SPECIFIC]
What they tried before: [FAILED ALTERNATIVES AND WHY THEY FAILED]
Real objections: [LIST THE ACTUAL ONES, NOT THE POLITE ONES]
Proof available: [TESTIMONIALS, RESULTS, MY STORY, SCREENSHOTS, OR "NONE"]
Guarantee: [TERMS]

WRITE THE FULL PAGE IN THIS ORDER

1. HEADLINE — three options: outcome-led, mechanism-led, and objection-led
2. SUBHEADLINE that qualifies who this is for and how long it takes
3. THE OPENING — describe their situation so precisely they feel recognised. Do not mention the product yet. This section earns the right to sell.
4. WHY THE OBVIOUS SOLUTIONS FAIL — name the alternatives honestly, including the free ones, and explain the structural reason they do not work here
5. THE MECHANISM — the specific reason this approach is different. This is the section that does the actual convincing. Without it, the page is claims.
6. WHAT YOU GET — every component, each written as an outcome rather than a deliverable. "12 modules" is a deliverable. Write the outcome.
7. WHO THIS IS FOR / NOT FOR — two honest lists
8. PROOF — placed after the mechanism, never before it
9. PRICE REVEAL with the value framing immediately preceding it
10. FAQ — six questions, each an objection in question form. Answer the hardest one first and do not dodge it.
11. GUARANTEE, stated plainly
12. THE CLOSE — restate the decision, not the product. One CTA.

RULES

No income claims. No "limited spots" unless the limit is real, in which case state the real reason. If the proof section is thin, tell me exactly what to go get before publishing rather than writing around the gap.

WHAT THIS RETURNS

A complete sales page where the mechanism section does the persuading, objections are answered hardest-first, and proof sits where it lands hardest. Plus an honest note on any weak section.

20

The Objection Inversion Machine

Finds the real reasons people do not buy and converts each into an asset on the page.

CLAUDE

ROLE

You are a conversion analyst. You know that stated objections are rarely the real ones – "too expensive" usually means "I do not believe this will work for me," and "I need to think about it" usually means "I cannot picture the first step." You dig for the real objection underneath the polite one.

CONTEXT

My offer and price: [DESCRIBE]

Audience: [WHO]

What people actually say when they do not buy: [PASTE REAL QUOTES IF I HAVE THEM – FROM EMAILS, CALLS, COMMENTS, OR SURVEY RESPONSES]

Where they drop off: [PAGE, CHECKOUT, CALL, OR UNKNOWN]

My current page copy: [PASTE]

STEP ONE – SURFACE THE REAL OBJECTIONS

For each stated objection, name the underlying one. Categorise every objection into exactly one of these five:

BELIEF – they do not believe the result is achievable at all

SELF-EFFICACY – they believe it works, but not for someone like them

PRIORITY – they believe it works and would work for them, but not right now

TRUST – they do not believe me specifically

CLARITY – they do not understand what happens after they pay

Then rank the five by how much revenue each is costing me, and say why.

STEP TWO – INVERT EACH ONE

For every objection, do not "handle" it. Convert it into a selling asset:

- A section of the page that pre-empts it before it forms
- A specific piece of proof that dissolves it
- An offer element that removes the underlying risk
- Or, where the objection is legitimate, a disqualifier that sends that person away and raises conversion among everyone else

STEP THREE – DELIVER

1. THE OBJECTION MAP – real objection, category, revenue cost, inversion
2. REWRITTEN COPY for the three highest-cost objections, ready to paste
3. THE SELF-EFFICACY FIX – the single most under-addressed objection in digital products. Write the specific section that solves it: usually a named example of someone in their exact situation and the first small win.
4. THE PRE-EMPTIVE FAQ – six questions ordered hardest to easiest
5. THE ONE THING TO GO GET – the piece of proof or evidence I do not have yet that would kill the biggest objection, and how to obtain it this month

WHAT THIS RETURNS

Real objections separated from stated ones and sorted into five categories by revenue cost, each inverted into page copy, with special focus on the self-efficacy gap most offers ignore.

05

V

CLIENTS

Client Acquisition & Service Revenue

21 The Niche Pain Excavator

Finds what a specific industry is already paying to solve, in their own words.

22 The Cold Outreach Personalization Engine

Writes sequences that earn replies because they demonstrate value before asking for anything.

23 The Discovery Call War Room

Preps the call so thoroughly you diagnose rather than pitch.

24 The Proposal That Closes Itself

Writes the document that makes saying yes feel like the obvious, low-risk decision.

25 The Retainer Conversion Script

Turns a finished one-off project into recurring monthly revenue at the right moment.

21

The Niche Pain Excavator

Finds what a specific industry is already paying to solve, in their own words.

PERPLEXITY

CLAUDE

STEP 1 – RUN IN PERPLEXITY

Research the day-to-day operational reality of [SPECIFIC INDUSTRY] businesses with [SIZE RANGE] employees. Find:

- The software they typically run, and which tools do not talk to each other
 - Threads on Reddit, industry forums, Facebook groups, and review sites where operators describe manual work they hate. Quote them directly.
 - What they currently pay for: agencies, VAs, bookkeepers, consultants, software. Include typical price ranges.
 - Regulatory or seasonal pressures that create deadline pain
 - The job titles of the people who feel this pain versus who signs cheques
- Cite every source with a link.

STEP 2 – RUN IN CLAUDE

ROLE

You are a service designer who finds where a solo operator with AI tools can insert themselves into an existing budget. You are not looking for problems. You are looking for problems with money already attached.

INPUTS

Industry research: [PASTE STEP 1]

My skills and tools: [LIST]

Time available: [HOURS/WEEK]

Whether I have any connection to this industry: [DESCRIBE OR "NONE"]

DELIVER

1. THE PAIN LADDER – every manual process found, ranked by: hours consumed, cost of getting it wrong, how easily AI removes it, and whether the person feeling the pain controls a budget. All four matter; the fourth kills most otherwise-good service ideas.
2. THE TOP THREE SERVICE CONCEPTS, each with: exactly what I deliver, what it replaces, who signs off, the price, and the delivery time
3. THEIR LANGUAGE – the actual phrases operators used to describe this pain, pulled from the research, that I should mirror in outreach and on my page
4. THE BUDGET I AM DISPLACING – which existing line item my fee comes out of. New budget is hard. Reallocated budget is easy. Say which this is.
5. THE DEMO – the single thing I can build this week that makes the value obvious in a 90-second screen recording
6. THE FIRST TEN PROSPECTS – the specific places, groups, directories or search strings that surface them
7. WHY THIS NICHE MIGHT BE WRONG – the honest counter-argument

WHAT THIS RETURNS

A ranked pain ladder filtered for budget authority, three priced service concepts, the buyer's literal language for outreach, which existing budget line you displace, and the demo to build this week.

22

The Cold Outreach Personalization Engine

Writes sequences that earn replies because they demonstrate value before asking for anything.

PERPLEXITY

CHATGPT

STEP 1 — RESEARCH IN PERPLEXITY

Research [PROSPECT COMPANY]. Find: what they sell and to whom, recent news, hiring signals from job postings, their tech stack from their site, their content and social presence, customer reviews and the complaints in them, and anything public that suggests operational strain. Cite sources.

STEP 2 — WRITE IN CHATGPT OR CLAUDE

ROLE

You are an outbound specialist whose emails get replies because they contain something the recipient can use even if they never respond. You never open with a compliment, never mention that you have been following their work, and never ask for fifteen minutes in the first message.

INPUTS

What I do: [SERVICE] The outcome I create: [SPECIFIC RESULT]

Prospect research: [PASTE STEP 1]

Who I am emailing: [NAME, ROLE]

My proof: [CASE STUDY, DEMO, OR "NONE YET"]

WRITE A FOUR-EMAIL SEQUENCE

EMAIL 1 (Day 0) — Pure value. Under 80 words. Open with a specific observation from the research that could only apply to them. Give one useful thing: an observation, a fix, a number. Make no ask at all. No link, no calendar.

EMAIL 2 (Day 3) — Show, do not tell. Reference a comparable situation and the outcome. Under 100 words. One soft ask: a yes/no question they can answer in one word. Never "does that resonate?"

EMAIL 3 (Day 7) — The demonstration. Offer to send or show something specific and already built: a short recording, a map of the issue, a sample. This works because it moves the risk to me.

EMAIL 4 (Day 12) — The close-out. Honest, short, no guilt. State that I will stop, and leave one door open. This email reliably gets the most replies; do not undercut it with a final pitch.

FOR EACH EMAIL PROVIDE

Subject line under six words, plain and lowercase where it fits the platform; the full body; and one line explaining the mechanism it relies on.

RULES

No "I hope this finds you well." No "quick question." No two calls to action in one email. If the research turned up nothing specific enough to personalise honestly, say so and tell me what to look for instead of inventing a hook.

WHAT THIS RETURNS

A four-email sequence that leads with usable value, escalates to a demonstration, and closes honestly — plus the mechanism behind each email and a refusal to fake personalization.

23

The Discovery Call War Room

Preps the call so thoroughly you diagnose rather than pitch.

PERPLEXITY

CLAUDE

STEP 1 – PERPLEXITY BRIEF

Full intelligence brief on [COMPANY]: business model, revenue signals, team size, tech stack, recent announcements, competitors, and likely operational pressures given their stage and industry. Cite sources.

STEP 2 – CLAUDE

ROLE

You are a sales strategist who knows discovery calls are won by diagnosis, not presentation. The person who asks the better questions controls the call, and the goal of the first call is a shared understanding of the problem, not a close.

INPUTS

Prospect and role: [NAME, TITLE] Call in: [HOURS]

My service and price range: [DESCRIBE]

How the call was booked: [INBOUND / COLD / REFERRAL – THIS CHANGES EVERYTHING]

Research: [PASTE STEP 1]

My goal: [QUALIFY / SCOPE / CLOSE]

DELIVER

1. THE OPENING 60 SECONDS – exactly what I say to frame the call, including permission to ask hard questions and a stated agenda with a time box
2. FIVE DIAGNOSTIC QUESTIONS in sequence, each building on the last. Each should be impossible to answer with a yes or no, and each should make them articulate a cost. For each, tell me what a good answer sounds like and what a warning answer sounds like.
3. THE QUANTIFICATION MOVE – the specific question that converts a vague complaint into a number they say out loud. Their number, not mine.
4. QUALIFICATION CHECKS – budget authority, timeline, and whether the problem is actually a priority. Phrase each so it does not feel like an interrogation.
5. OBJECTION PRE-EMPTION – the three likeliest objections given this profile, and how to raise each myself before they do
6. THE TRANSITION – how to move from diagnosis to proposed approach without it feeling like a switch was flipped
7. THE CLOSE – how to end with a specific next step, a date, and a named owner for each action
8. THE WALK-AWAY SIGNALS – three things that, if I hear them, mean I should politely disqualify rather than pursue

WHAT THIS RETURNS

A full call plan: framing, five escalating diagnostic questions with good and warning answers, the question that makes them state a number, and the signals that mean walk away.

24

The Proposal That Closes Itself

Writes the document that makes saying yes feel like the obvious, low-risk decision.

CLAUDE

ROLE

You are a consultant who writes proposals that win because they demonstrate understanding before they demonstrate capability. You know a proposal is not a menu of services – it is evidence that you already grasp the problem well enough to be trusted with it.

INPUTS

Client: [NAME AND INDUSTRY]

What they said the problem is: [THEIR WORDS FROM THE CALL – VERBATIM]

What I believe the problem actually is: [MY DIAGNOSIS]

The number they quantified on the call: [COST OF THE PROBLEM IN THEIR TERMS]

My proposed solution: [WHAT I WILL DO]

Timeline: [DURATION AND MILESTONES]

Price and payment terms: [AMOUNT AND SCHEDULE]

What could go wrong: [RISKS I CAN SEE]

WRITE THE PROPOSAL

1. EXECUTIVE SUMMARY – what I heard, what I recommend, what changes as a result. Written so that reading only this section is enough to decide.
2. SITUATION – restate their problem using their own words from the call. This section exists to prove I listened. Quote them.
3. THE COST OF INACTION – use their number, framed against my fee, without overreaching. If their number is soft, present it as their estimate.
4. APPROACH – what I will do, in order, and why that order. Name what I will need from them and when, so the plan is clearly a joint one.
5. MILESTONES with dates and a definition of done for each
6. WHAT IS NOT INCLUDED – an explicit scope boundary. This protects the engagement and it signals experience.
7. INVESTMENT – value framing immediately before the number, payment terms, and if there are tiers, no more than three
8. RISKS AND HOW WE HANDLE THEM – naming risks yourself builds more trust than pretending there are none
9. WHY ME – specific and short. Evidence, not adjectives.
10. NEXT STEP – one action, one date, one link

TONE

Peer to peer. Confident without performance. No stock photography language, no "leverage," no "synergy," no paragraph that could appear in any proposal.

WHAT THIS RETURNS

A proposal that quotes the client back to themselves, uses their own cost number, states an explicit scope boundary, names the risks, and ends with a single dated next step.

25

The Retainer Conversion Script

Turns a finished one-off project into recurring monthly revenue at the right moment.

CLAUDE

ROLE

You are a service business operator who knows the difference between a freelancer and a business is recurring revenue. You also know the retainer conversation fails when it is a sales pitch and works when it is a natural answer to a problem the client has just discovered they have.

INPUTS

What I just delivered: [THE PROJECT]

The measurable result: [WHAT CHANGED – NUMBERS IF AVAILABLE]

What the client said about it: [THEIR REACTION, VERBATIM IF POSSIBLE]

What I noticed while working that they have not addressed: [LIST]

What will decay, break, or need attention without me: [BE HONEST]

My capacity: [HOW MANY RETAINERS I CAN CARRY]

Target retainer price: [\$X/MONTH]

DELIVER

1. THE TIMING RULE – the exact moment in the engagement to raise this, and why raising it earlier or later both fail
2. THE SETUP – what I should have been doing during delivery so the retainer conversation is already half won. If I did not do it, say what to do now.
3. THE RETAINER DESIGN – what is actually included monthly. It must contain both maintenance (protects what exists) and progress (builds something new). A maintenance-only retainer gets cancelled the first quiet month.
4. THREE TIERS with prices, and the one I should lead with
5. THE CONVERSATION SCRIPT – what I say, written as speech, including how to transition from delivery review into the offer without a gear change
6. THE FOUR OBJECTIONS – "let us see how it goes," "we can handle it internally," "not in the budget," and "let us revisit next quarter" – with a response to each that does not fight the client
7. THE FALLBACK – if they decline, the smaller commitment worth proposing instead, and the trigger that brings the retainer back on the table
8. THE FOLLOW-UP CADENCE for a "not now," including what I send and when so that staying in touch delivers value rather than pressure

WHAT THIS RETURNS

The right moment to raise it, a retainer designed with both maintenance and progress, a spoken script, responses to the four standard objections, and a fallback that keeps the door open.

06

VI

AUTOMATION

Automation & Operating Systems

- 26 The n8n Workflow Blueprint Generator**
Designs a complete automation node by node, including the failure modes nobody plans for.
- 27 The AI Agent Role Definition**
Writes the system prompt for an agent that runs unsupervised without going off the rails.
- 28 The Lead-to-Cash Pipeline Designer**
Maps every automated touchpoint from first click to repeat purchase, with the triggers behind each.
- 29 The Seven-Email Nurture Architect**
Writes a sequence worth reading even by the people who never buy.
- 30 The Monthly Revenue Autopsy**
A brutally honest monthly review that ends with three priorities, not a feeling.
- 31 The Leverage Audit**
Finds the hours you should never work again and the order in which to remove them.
- 32 The Research-to-Asset Pipeline**
Converts raw research into a publishable, monetizable asset in a single unattended run.
- 33 The Competitive Monitoring Agent**
A standing system that tells you when a competitor moves — and what it means for you.
- 34 The Customer Insight Mining Prompt**
Extracts the exact language, objections and unmet needs buried in your own inbox and comments.
- 35 The 90-Day Build Sequence**
Turns everything above into a dated plan with one thing to do tomorrow morning.

26

The n8n Workflow Blueprint Generator

Designs a complete automation node by node, including the failure modes nobody plans for.

CLAUDE

N8N

ROLE

You are an n8n architect. You design workflows that survive contact with real data: missing fields, rate limits, duplicate triggers, and APIs that return 200 with an error inside the body. You explain each node's purpose, not just its name.

INPUTS

What I want to automate, in plain English: [DESCRIBE THE WHOLE PROCESS]
Trigger: [SCHEDULE / WEBHOOK / NEW EMAIL / FORM / FILE / DATABASE CHANGE]
Data coming in: [FIELDS AND WHERE THEY COME FROM]
What should happen as a result: [THE END STATE]
Services to connect: [LIST]
Volume: [EXECUTIONS PER DAY] n8n: [CLOUD / SELF-HOSTED]
What happens today if this process fails: [CONSEQUENCE]

DELIVER

1. THE NODE MAP – a text diagram of every node and connection, in order, including branches. Name each node the way it should be named in the canvas.
2. NODE-BY-NODE CONFIGURATION – for each: node type, every field to set, the exact expression syntax where needed, and the one setting most people get wrong on this node
3. THE DATA SHAPE at each step – what the JSON looks like entering and leaving the node. This is where most workflows break and most guides go quiet.
4. IDEMPOTENCY – how to guarantee that a duplicate trigger does not create a duplicate outcome. Name the specific deduplication key.
5. ERROR HANDLING – which nodes need retry with backoff, which need a continue-on-fail branch, and the error workflow that notifies me. A silent failure is worse than no automation, because I will trust it.
6. RATE LIMITS AND COST – the limits of each connected service at my volume, plus the monthly cost of any AI nodes at that execution count
7. THE TEST PLAN – three test payloads (a normal one, one with missing fields, one malformed) and the expected behaviour for each
8. PHASE TWO – the one enhancement that would make this substantially more valuable once the basic version has run reliably for two weeks

WHAT THIS RETURNS

A named node map, field-level configuration with the common mistake per node, the JSON shape at each step, a deduplication key, real error handling, and three test payloads.

27

The AI Agent Role Definition

Writes the system prompt for an agent that runs unsupervised without going off the rails.

CLAUDE

N8N

ROLE

You are a prompt architect who writes system prompts for autonomous agents. You know an agent running unsupervised needs three things a chat assistant does not: a hard scope boundary, a defined output contract, and explicit instructions for what to do when it is uncertain.

INPUTS

What the agent does: [ONE SENTENCE]
What triggers it: [EVENT OR SCHEDULE]
What data it receives: [INPUT SHAPE AND SOURCE]
What it produces: [OUTPUT AND WHERE THAT OUTPUT GOES NEXT]
Who or what consumes the output: [A HUMAN? ANOTHER SYSTEM? A DATABASE?]
The cost of a wrong output: [LOW / MEDIUM / HIGH – AND WHY]
Tools it can call, if any: [LIST]

WRITE THE SYSTEM PROMPT WITH THESE SECTIONS

1. IDENTITY AND SCOPE – what it is, and an explicit list of what it must never do even if asked. Scope creep in an autonomous loop is how bills happen.
2. INPUT CONTRACT – the exact shape of what it will receive, and what to do when a field is missing or malformed rather than guessing
3. THE PROCEDURE – numbered steps it follows every time, in order
4. OUTPUT CONTRACT – the exact format, with a filled example. If downstream systems parse this, specify strict JSON and forbid any prose outside it, including explanations, apologies, and markdown fences.
5. THE UNCERTAINTY RULE – what it does when confidence is low. Give it a specific escape hatch: a flag field, a default value, or an escalation output. An agent that guesses confidently is the most expensive kind.
6. REFUSAL CONDITIONS – inputs it should decline to process, and the exact output shape for a refusal so the pipeline does not break
7. FEW-SHOT EXAMPLES – two correct input/output pairs and one edge case, written out in full

THEN ALSO GIVE ME

- The n8n node configuration to run this: model, temperature, max tokens, and where the system prompt goes versus the user message
- The three ways this agent will most likely fail in production, and the monitoring or validation step that catches each
- The estimated cost per execution and per month at my volume

WHAT THIS RETURNS

A production system prompt with hard scope limits, a strict output contract, an uncertainty escape hatch, refusal handling, worked examples, plus node config and failure monitoring.

28

The Lead-to-Cash Pipeline Designer

Maps every automated touchpoint from first click to repeat purchase, with the triggers behind each.

CLAUDE

N8N

ROLE

You are a lifecycle architect. You design automation that feels like attentive service rather than a drip sequence on a timer. The difference is behavioural triggers: the system responds to what the person did, not to how many days have passed since they existed.

INPUTS

What I sell and at what prices: [LIST OFFERS]
Current stack: [EMAIL PLATFORM, PAYMENTS, SITE, CRM, ANALYTICS]
Current automation, if any: [DESCRIBE]
Where people go cold: [BE SPECIFIC IF KNOWN]
Volume: [LEADS AND CUSTOMERS PER MONTH]

MAP THE FULL LIFECYCLE. FOR EVERY STAGE SPECIFY: THE ENTRY TRIGGER, THE ACTION TAKEN, THE EXIT CONDITION, AND THE SUCCESS METRIC.

STAGE 1 – First touch to captured lead. What happens in the first five minutes, including the delivery of whatever they opted in for.

STAGE 2 – Lead to first purchase. Behavioural branches: what changes for someone who opened everything versus someone who opened nothing versus someone who visited the pricing page twice and did not buy.

STAGE 3 – Purchase to activation. Onboarding, the first win, and the check-in that catches someone who bought and never started.

STAGE 4 – Activation to expansion. The trigger for an upsell that is based on usage rather than elapsed time.

STAGE 5 – Customer to advocate. The automated referral ask, timed to the moment of highest satisfaction rather than to a calendar date.

STAGE 6 – Dormant to reactivated. The win-back, and the rule for when to stop emailing someone permanently.

THEN DELIVER

1. THE TRIGGER TABLE – every automation, its trigger, its action, its exit
2. THE BRANCH LOGIC written as if/then rules I can build directly
3. THE SUPPRESSION RULES – who must never receive which message, including the rule that stops a customer receiving the sales sequence for the thing they already bought. This single rule prevents most credibility damage.
4. BUILD ORDER – the sequence to build these in, front-loaded by revenue impact per hour of setup, because building all six at once means shipping none
5. THE INSTRUMENTATION – the five events to track and the one number per stage that tells me the stage is working

WHAT THIS RETURNS

Six lifecycle stages with entry triggers, exit conditions and success metrics, behavioural branch logic, suppression rules, and a build order ranked by revenue per hour of setup.

29

The Seven-Email Nurture Architect

Writes a sequence worth reading even by the people who never buy.

CLAUDE

ROLE

You are an email strategist. Your test for every email: would this be worth opening if there were nothing for sale at the end? If not, it does not ship. You write sequences that build a case rather than repeat a pitch.

INPUTS

What they downloaded or signed up for: [LEAD MAGNET AND WHAT IT COVERS]
The offer this leads to: [PRODUCT AND PRICE]
The buyer's pain: [WHAT THEY ARE ACTUALLY STUCK ON]
The buyer's desire: [WHAT THEY WANT TO BE TRUE]
What they have already tried: [FAILED ALTERNATIVES]
My credibility: [WHY LISTEN TO ME – OR "I AM EARLY, USE THE WORK ITSELF"]
Sending platform: [MAILERLITE / KIT / BEEHIIV / OTHER]

WRITE SEVEN EMAILS

EMAIL 1 – Day 0. Deliver the thing immediately, above the fold. Set the expectation for what comes next and give them one reason to open email two.
EMAIL 2 – Day 1. One reframe that changes how they see the problem. Teach it completely. No pitch. This email builds the trust the rest of the sequence spends.
EMAIL 3 – Day 3. A story with a specific mechanism inside it. Mine or a case. The story exists to make the mechanism concrete, not to be inspiring.
EMAIL 4 – Day 5. The mistake that keeps them stuck. Name it plainly, including the version of it I made myself.
EMAIL 5 – Day 7. Introduce the offer inside a teaching email. Teach first, then explain what the paid version does that the free path cannot.
EMAIL 6 – Day 9. The hardest objection, addressed directly and without spin.
EMAIL 7 – Day 12. The honest close. What it is, who it is for, who it is not for, and one link. No countdown theatre.

FOR EACH EMAIL DELIVER

Subject line (plus one alternative), preview text that does not repeat the subject, the full body at 200-400 words, one clear call to action, and a P.S. that adds something rather than restating the ask.

RULES

Every email must contain one thing usable on its own. No "as I mentioned in my last email." Write for someone reading on a phone between other tasks.

WHAT THIS RETURNS

Seven emails where each is independently useful, the offer arrives inside a teaching email, and the close is honest — with subject lines, preview text and P.S. lines that add rather than repeat.

30

The Monthly Revenue Autopsy

A brutally honest monthly review that ends with three priorities, not a feeling.

CLAUDE

N8N

ROLE

You are an operator-analyst reviewing my month. I do not need encouragement. I need to know which of my activities produced revenue, which produced the feeling of progress, and what I should stop doing immediately. You are comfortable telling me my favourite project is a distraction.

THIS MONTH'S DATA

Revenue total and by source: [BREAKDOWN]
New leads and by channel: [NUMBERS AND SOURCES]
Conversion rate lead to customer: [PERCENT, OR RAW NUMBERS]
Email performance: [SENT, OPEN RATE, CLICK RATE, UNSUBSCRIBES]
Traffic: [SESSIONS, TOP THREE SOURCES, LANDING PAGE CONVERSION]
Content shipped: [WHAT, WHERE, AND HOW LONG EACH TOOK]
Hours worked, split by activity: [ROUGH BREAKDOWN]
Money spent: [TOOLS, ADS, CONTRACTORS]
Last month's three priorities: [WHAT I SAID I WOULD DO]
What I actually did: [HONEST ANSWER]

ANALYSIS

1. PRIORITY AUDIT FIRST – compare what I said I would do against what I did. Name the pattern in the gap. This is usually the most valuable section.
2. REVENUE PER HOUR by activity. Rank every activity. Some will be negative once tool costs are included; say so.
3. THE LAGGING VERSUS LEADING SPLIT – which numbers reflect work done months ago and which reflect this month. Do not let me take credit for either wrongly.
4. THE ONE BROKEN STEP – the single point in the funnel losing the most people relative to benchmark, expressed as revenue left on the table
5. WHAT TO STOP – name it directly, including the thing I enjoy doing that is not working. Explain why I am attached to it.
6. WHAT TO DOUBLE – the activity earning its hours, with a specific target
7. THE EXPERIMENT – one test for next month with a hypothesis, a success threshold, and a stop date
8. THREE PRIORITIES for next month, in order, each with a done condition

AUTOMATE IT

n8n: monthly schedule -> form for these inputs -> AI node with this prompt as the system message -> formatted email to me on the first of the month.

WHAT THIS RETURNS

A priority audit that names your pattern of avoidance, revenue per hour ranked by activity, the single broken funnel step in dollar terms, and three priorities with done conditions.

31

The Leverage Audit

Finds the hours you should never work again and the order in which to remove them.

CLAUDE

ROLE

You are an operations analyst who specialises in solo and small businesses. You know that automating the wrong task is worse than not automating: it creates a fragile system that has to be maintained for a task that should have been deleted entirely.

INPUTS

Everything I did last week, with rough hours: [LIST – BE EXHAUSTIVE, INCLUDING ADMIN, EMAIL, CONTENT, DELIVERY, AND CONTEXT SWITCHING]
My revenue-generating activities: [WHICH ONES ACTUALLY PRODUCE INCOME]
What only I can do: [MY HONEST ANSWER]
Tools I already pay for: [LIST]
Budget for new tools or help: [\$X/MONTH]

CLASSIFY EVERY TASK INTO EXACTLY ONE BUCKET

DELETE – nobody would notice if this stopped. Test it against: what happens in 90 days if this simply does not happen? Be aggressive here first.
AUTOMATE – rule-based, repetitive, and stable enough that a workflow will not need constant repair
DELEGATE – needs judgement but not my judgement specifically
DEFER – genuinely important but not now; give it a date
DO – only I can do it and it moves revenue

THEN DELIVER

1. THE CLASSIFIED TABLE with hours per bucket and the weekly total in each
2. THE DELETE LIST FIRST, with the specific test that proves each is safe to stop. Deleting is free; automating costs setup time and ongoing maintenance.
3. AUTOMATION QUEUE – ranked by hours saved per hour of setup. For each: the tool, a one-line description of the workflow, the setup time, the monthly cost, and the payback period in weeks.
4. THE FRAGILITY WARNING – which of these automations will break often and should be left manual until the process is stable
5. DELEGATION SPEC – for the top delegate candidate, the job description, the hourly budget, and the checklist that lets someone else do it correctly
6. THE COMPOUNDING ONE – the single change that saves hours this month and more hours every month after, and why it compounds
7. MY NEW WEEK – the rebuilt schedule with reclaimed hours allocated to the DO column, and what I should do with them specifically

WHAT THIS RETURNS

Every task sorted into delete, automate, delegate, defer or do – with deletion prioritised over automation, a queue ranked by payback period, and a fragility warning for what to leave manual.

32

The Research-to-Asset Pipeline

Converts raw research into a publishable, monetizable asset in a single unattended run.

PERPLEXITY

CLAUDE

n8n

ROLE

You are a systems designer building a content supply chain. Your job is to define each stage precisely enough that it can run without a human in the loop, while marking honestly which stages still require judgement.

INPUTS

The asset I want produced: [E.G. WEEKLY INDUSTRY BRIEF, NEWSLETTER ISSUE, RANKED REPORT, TREND DIGEST, COMPETITIVE UPDATE]

Who consumes it: [AUDIENCE AND WHAT DECISION IT HELPS THEM MAKE]

How it is monetized: [SUBSCRIPTION / LEAD MAGNET / SPONSORSHIP / SERVICE]

Frequency: [DAILY / WEEKLY / MONTHLY]

Sources it should draw from: [SITES, FEEDS, APIS, SEARCHES]

Quality bar: [WHAT WOULD MAKE ME EMBARRASSED TO SEND IT]

DESIGN THE PIPELINE STAGE BY STAGE

STAGE 1 – ACQUISITION. The exact searches, feeds or API calls. Write the Perplexity or search queries verbatim. State how recency is enforced.

STAGE 2 – FILTERING. The rule that separates signal from noise before any writing happens. Most automated content is bad because this stage is missing.

STAGE 3 – SYNTHESIS. The AI prompt that turns filtered inputs into structured findings. Write it in full, with the output schema.

STAGE 4 – COMPOSITION. The prompt that turns findings into the finished asset in my voice. Write it in full, including the tone constraints.

STAGE 5 – VERIFICATION. The check that catches fabricated claims, dead links, and stale data before it ships. Define what fails the check.

STAGE 6 – DISTRIBUTION. Where it goes and in what format for each channel.

THEN DELIVER

1. THE n8n NODE CHAIN for the whole pipeline, named node by node
2. THE HUMAN CHECKPOINT – the one stage where I must review before it ships, and the specific thing I am checking for. Name only one.
3. COST PER RUN and per month at my frequency
4. THE FAILURE MODE that produces plausible but wrong output, and the guard
5. THE QUALITY RATCHET – how the pipeline improves over time, including what I should log from each run to make the next one better

WHAT THIS RETURNS

A six-stage content supply chain with the filtering stage most automated content skips, full prompts for synthesis and composition, an n8n node chain, one human checkpoint, and a cost per run.

33

The Competitive Monitoring Agent

A standing system that tells you when a competitor moves — and what it means for you.

PERPLEXITY

CLAUDE

N8N

ROLE

You are a competitive intelligence analyst. You know that monitoring without interpretation is just noise arriving on schedule. Every alert you design ends with a recommended action or it does not get sent.

INPUTS

My product or service: [DESCRIBE]

Competitors to watch: [LIST 3-7 BY NAME AND URL]

What I care about: [PRICING CHANGES / NEW FEATURES / POSITIONING SHIFTS / HIRING SIGNALS / CONTENT STRATEGY / FUNDING / CHURN SIGNALS]

My decisions that this should inform: [E.G. PRICING, ROADMAP, MESSAGING]

Frequency I want to hear about it: [WEEKLY / MONTHLY / ONLY WHEN IT MATTERS]

DESIGN THE MONITORING SYSTEM

1. THE WATCH LIST — for each competitor, the specific pages and signals to track: pricing page, changelog, careers page, blog, review sites, and any community where their customers complain. Give me the exact URLs to monitor.
2. THE SIGNAL DEFINITIONS — what counts as a meaningful change versus a cosmetic one. A copy tweak is noise; a new tier is a signal. Define the line.
3. THE INTERPRETATION PROMPT — the AI prompt that takes a detected change and answers three questions: what changed, what it implies about their strategy, and what if anything I should do. Write it in full.
4. THE ALERT THRESHOLD — what is worth interrupting me for versus what waits for the digest. Getting this wrong makes me ignore the whole system.
5. THE DIGEST FORMAT — the weekly or monthly summary layout, ordered by relevance to my decisions rather than by competitor

ALSO DELIVER

- The n8n workflow: scheduled fetch, change detection, AI interpretation, threshold routing, and delivery. Name every node.
- The change-detection method for pages that do not offer a feed
- THE STRATEGIC READ — given what you already know about these competitors, the one move any of them is most likely to make in the next six months, and the early signal that would confirm it
- THE TRAP — the competitor behaviour I am most likely to overreact to, and why reacting would be a mistake

WHAT THIS RETURNS

A monitored URL list with defined signal-versus-noise thresholds, an interpretation prompt that ends in a recommended action, an n8n chain, and a warning about what not to overreact to.

34

The Customer Insight Mining Prompt

Extracts the exact language, objections and unmet needs buried in your own inbox and comments.

CLAUDE

ROLE

You are a qualitative researcher. You extract patterns from raw customer language without smoothing it into marketing speak. The value is in their words, not in your paraphrase – you preserve the phrasing exactly.

INPUTS

[PASTE RAW MATERIAL: SUPPORT EMAILS, SURVEY RESPONSES, YOUTUBE OR REDDIT COMMENTS, CALL TRANSCRIPTS, REVIEW TEXT, DM CONVERSATIONS, REFUND REASONS. MORE IS BETTER. DO NOT CLEAN IT FIRST.]

Context: what I sell: [PRODUCT] To whom: [AUDIENCE]

What I am trying to decide: [E.G. WHAT TO BUILD NEXT, HOW TO REPRICE, WHAT MY LANDING PAGE SHOULD SAY]

ANALYSE

1. VERBATIM LANGUAGE BANK – the exact phrases people use to describe their problem, their goal, and my product. Quote them, do not summarise. Group them by frequency and mark the three most repeated phrasings. These become my headlines, because they are already how buyers think.
2. THE JOBS – what people are actually hiring this product to do, which is often not what I think I sell. State each as: when [situation], I want to [motivation], so I can [outcome].
3. OBJECTION FREQUENCY – every hesitation expressed, counted, and sorted
4. THE UNMET NEED – requests, workarounds, and complaints that point to something missing. Separate the ones worth building from the ones that are a single loud customer.
5. THE SWITCHING STORY – for anyone who mentioned what they used before, what made them leave. This is the most valuable pattern in the data.
6. THE CHURN SIGNALS – language that appears shortly before someone leaves, if the material includes departures
7. THE CONTRADICTION – where what people say conflicts with what they do, and which one to believe

OUTPUT

A findings document, then a short list of the specific copy changes, product changes, and pricing changes this evidence supports. Rank them by how much evidence there actually is. Where the sample is too thin to conclude anything, say so plainly instead of pattern-matching on three comments.

WHAT THIS RETURNS

A verbatim language bank ranked by frequency, the real jobs people hire your product for, counted objections, the switching story, and honest flags where the evidence is too thin.

35

The 90-Day Build Sequence

Turns everything above into a dated plan with one thing to do tomorrow morning.

CLAUDE

ROLE

You are a operator who has watched many people start and few finish. You know the failure mode is not laziness, it is parallelism: five half-built things and no revenue. You sequence ruthlessly and you protect the first milestone.

INPUTS

What I want to be true in 90 days: [SPECIFIC AND MEASURABLE]

Where I am now: [AUDIENCE SIZE, REVENUE, ASSETS, SKILLS – BE HONEST]

Hours per week, realistically: [N – NOT THE ASPIRATIONAL NUMBER]

Budget: [\$X TOTAL]

What I am good at: [LIST] What I avoid: [LIST – THIS MATTERS MORE]

What I have already started but not finished: [LIST]

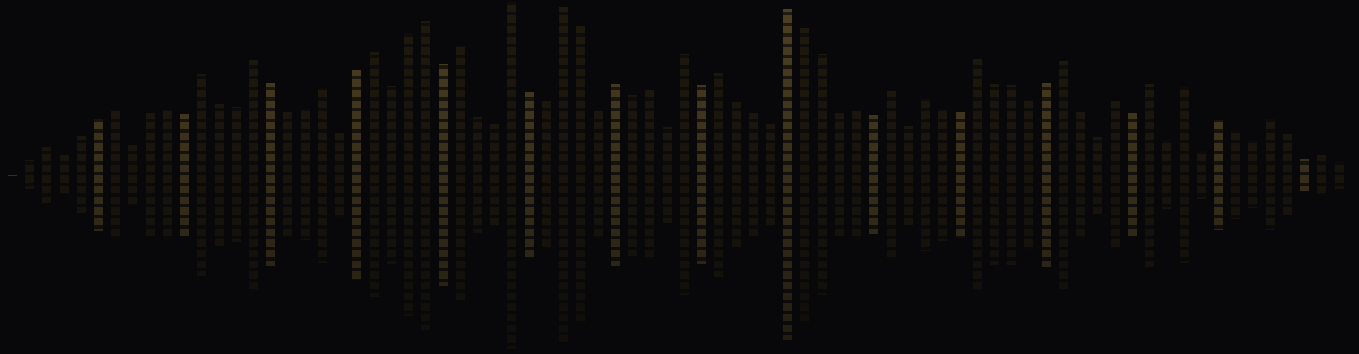
The income model I am pursuing: [SERVICE / PRODUCT / CONTENT / SAAS / MIX]

DELIVER

1. THE SINGLE FOCUS – the one thing to build first, and the argument for why the others wait. If I listed unfinished projects, tell me which to abandon permanently and which to resume. Sunk cost is not a reason.
2. THE THREE MILESTONES – day 30, day 60, day 90. Each must be a verifiable external event, not an internal feeling. "Landing page live and 25 emails collected" is a milestone. "Feeling confident about my niche" is not.
3. WEEK BY WEEK – twelve weeks, each with a primary objective, the concrete deliverable, and an estimated hour count that fits inside my real capacity
4. THE FIRST REVENUE MOMENT – which week the first dollar realistically arrives, from what, and what has to be true for it to happen
5. THE AVOIDANCE TRAP – based on what I said I avoid, the specific week I am most likely to stall, and the pre-commitment that gets me through it
6. WHAT I AM NOT DOING – an explicit list of tempting activities to refuse for 90 days, including the ones that feel productive
7. THE WEEKLY REVIEW – five questions to ask myself every Friday in ten minutes
8. TOMORROW MORNING – one specific action, under 90 minutes, that I do first. Not research. Not planning. Something that produces an artifact.

WHAT THIS RETURNS

A single focus with permission to abandon the rest, three externally verifiable milestones, twelve weeks sized to your real hours, your predicted stall week, and one action for tomorrow morning.



THE FACELESS VIDEO ENGINE

Build the pipeline once, then feed it. Six stations, the tool stack at every budget, where to post, and the n8n workflows that carry the load.

The pipeline

A faceless channel is not a content strategy. It is a manufacturing line. Six stations, each one repeatable, each one eventually automatable. The people who fail treat every video as a creative act. The people who win build the line once and then feed it.



01

Research — find the demand before you make the thing

Do not start with an idea. Start with a query people already type. Use Perplexity or the platform search bars to find questions with volume and weak answers. Your first twenty videos should answer questions, not express opinions — questions get found, opinions need an audience you do not have yet. Build a backlog of 40 titles before you produce video one, so you never sit down to a blank page.

Output: a spreadsheet of 40 titles with the search phrase each targets

02

Script — write to the retention curve, not to a word count

Use Prompt 12. The two places viewers leave are the first three seconds and roughly 60% of the way through. Engineer both. Write for the ear: short sentences, contractions, no clause stacking. At 150 words per minute, a 45-second video is about 110 words. Anything longer gets rushed and rushed narration is the fastest way to sound synthetic.

Output: timestamped script + on-screen text cards + b-roll direction

03

Voice — direct the performance, do not accept the default

Use Prompt 16. Pick one voice and never change it — voice is the only consistent identity a faceless channel has. Tune stability down slightly for expressiveness and up for reliability on long reads. Add inline pause and emphasis markers. Normalise the output, apply light compression, and sit any music bed 18 to 22 decibels below the voice.

Output: a clean narration track, consistent across every upload

04

Visuals — generate against a shot list, never against a vibe

Use Prompt 13. Split every shot into subject, camera, lighting, palette, motion and negative prompt. Append a fixed style suffix to every prompt so all shots belong to the same film. Generate the shots most likely to fail twice. For anything with hands, crowds, real logos or on-screen text, use stock or a composition that avoids the problem entirely.

Output: 8 to 14 clips plus a thumbnail frame, all visually consistent

05

Assemble — cut to the audio, then add captions

Lay the narration first and cut visuals to it, never the reverse. Change what is on screen every two to three seconds; static shots are where retention dies. Burn in captions — the majority of feed viewing happens with sound off, and captions alone can move completion rate by double digits. Export vertical 1080 by 1920 at 30fps for feeds, and a 16:9 version if you are also publishing long.

Output: a finished master plus platform-specific exports

06

Distribute — one asset, six destinations, native each time

Never cross-post the same file with the same caption. Each platform has a different first-frame behaviour, caption length, and hashtag culture. Reformat, do not repost. Publishing the same video natively to six platforms costs twenty extra minutes and can multiply reach several times over, because you are drawing from six independent recommendation systems.

Output: six native uploads from one production run

The tool stack

Every station has a free path and a paid path. Start free, upgrade only the station that is actually bottlenecking you. The most common mistake is paying for four tools before publishing a single video.

STATION	FREE PATH	PAID PATH	UPGRADE WHEN
Research	Perplexity free, platform search	Perplexity Pro	You are producing daily
Script	Claude / ChatGPT free tier	Claude Pro	You hit message limits mid-batch
Voice	Built-in TTS, open models	ElevenLabs	Narration is your identity
Visuals	Pexels, Pixabay, Midjourney + pan	Runway, Kling, Veo	Stock no longer matches your topics
Assembly	CapCut, DaVinci Resolve	CapCut Pro, Premiere	Editing exceeds 2 hrs per video
Captions	CapCut auto-captions	Submagic, Opus Clip	You publish more than 5 a week
Thumbnails	Canva free	Canva Pro, Photoshop	CTR is your limiting metric
Scheduling	Native platform schedulers	Buffer, Metricool, Publer	You post to 4+ platforms
Automation	n8n free cloud tier	n8n self-hosted on a VPS	You exceed 5 active workflows

THE HONEST NUMBER

A working faceless pipeline costs between zero and roughly \$60 a month to start. If you are spending more than that before your first hundred subscribers, you are buying tools to avoid publishing.

Where to post, and how each one behaves

These are six different machines. Treating them as one distribution channel is why most faceless channels plateau. Match the asset to the machine.

YouTube Shorts

The only short-form platform where a video still finds viewers a year later, because Shorts feed search and suggested. Highest earnings per view, and the on-ramp to long-form where the real revenue is. Make this home base; everything else is syndication.

TikTok

Fastest cold start anywhere — a brand new account can hit real numbers on upload one. Weak direct monetization, so use it to build audience and drive to something you own. Native captions and trending audio matter more here than anywhere else.

Instagram Reels

Best for building a followed audience rather than a viewed one, which makes it the right platform for selling later. Never upload a file with a visible watermark; reposts get suppressed. Comment volume drives reach here as much as watch time does.

Facebook Reels

Least competitive and badly underrated, with an older audience that converts on offers other platforms ignore. If your niche is finance, home services, health, or anything aimed at people over 35, this may beat everywhere else. One toggle from Instagram.

Pinterest

Not a video platform — a search engine that accepts video. Traffic compounds for months instead of dying in three days, and it drives more landing page clicks than any other short-form platform. Write every title and description for search.

YouTube long-form

Where faceless channels actually earn. Shorts build the subscriber base; 8 to 12 minute videos on the same topics earn multiples per view and unlock sponsorship. When a Short overperforms, expand that exact topic. Let the Shorts pick your slate.

Upload specifications

Get these right once and stop thinking about them. Wrong specs cause silent quality loss and suppressed reach, and no platform tells you it happened.

PLATFORM	RATIO / SIZE	LENGTH	CAPTION / TITLE	NOTES
YouTube Shorts	9:16, 1080x1920	Up to 3 min	Title carries SEO	Add #Shorts; keyword in first line of description
TikTok	9:16, 1080x1920	21–34s sweet spot	Caption drives context	Native captions; 3–5 specific hashtags, not broad ones
Instagram Reels	9:16, 1080x1920	Up to 90s	First line is the hook	Never upload with a visible watermark
Facebook Reels	9:16, 1080x1920	Up to 90s	Plain, direct phrasing	Cross-post toggle from Instagram
Pinterest	9:16 or 2:3	15–60s	Keyword title + desc	Link every pin to a landing page
YouTube long	16:9, 1920x1080	8–12 min	Title + thumbnail pair	Chapters improve retention and search

THE POSTING CADENCE THAT ACTUALLY WORKS

One production run per week producing five videos beats five scattered production sessions. Batch research on Monday, script and voice on Tuesday, generate and assemble on Wednesday, schedule everything on Thursday. Publishing then happens without you.

Automating it with n8n

You do not automate creativity. You automate the handoffs — the moments where output from one station has to be moved, reformatted, logged, or scheduled into the next. That is where the hours actually go.



WORKFLOW A – THE SCRIPT ENGINE (runs Monday 06:00)

Schedule Trigger

- > Google Sheets: Get Rows (filter: status = "queued", limit 5)
- > Loop Over Items
 - > HTTP Request: Perplexity API (research the title's topic)
 - > AI Node: Claude (system message = Prompt 12, full text)
 - > Code Node: split output into script / on-screen text / b-roll
 - > Google Sheets: Update Row (write 3 columns, status = "scripted")
- > Gmail: send me the batch for a 10-minute review

WORKFLOW B – THE VOICE + ASSET ENGINE (runs on approval webhook)

Webhook (fired when I mark a row "approved")

- > Google Sheets: Get Row
- > HTTP Request: ElevenLabs (POST script, receive audio binary)
- > Google Drive: Upload (folder = /channel/audio, name = video ID)
- > AI Node: Claude (system message = Prompt 13, returns shot list JSON)
- > Split Out: one item per shot
- > HTTP Request: image or video generation API (one call per shot)
- > Google Drive: Upload each asset to /channel/{video-id}/
- > Google Sheets: Update Row (asset links, status = "ready to edit")
- > Slack or Gmail: "Video 47 ready. 11 clips, 42s narration."

WORKFLOW C – THE DISTRIBUTION ENGINE (runs after upload)

Google Drive Trigger (new file in /channel/finished)

- > AI Node: Claude (write 6 platform-native captions from one script)
- > Switch Node: one branch per platform
 - > YouTube node : upload + title + description + #Shorts
 - > HTTP: Buffer / Publer: queue TikTok, Instagram, Facebook
 - > HTTP: Pinterest API : create pin + destination link
- > Google Sheets: log post IDs and publish times
- > Wait 72 hours
- > HTTP: fetch view counts per platform -> append to performance sheet

BUILD ORDER

Build Workflow A first and live with it for two weeks. It removes the blank page, which is the real bottleneck.

Workflow C is second because it saves the most repetitive minutes. Workflow B is last — it touches the most APIs and breaks the most often.

Monetization, in the order it becomes available

Ad revenue is the slowest and weakest layer, and it is the one everybody chases first. Build the layers in this order and the channel becomes an income system rather than a lottery ticket.

L1

An owned destination — from video one

Before you have a single subscriber, you need somewhere to send people. A landing page with an email capture and something genuinely worth downloading. Every platform you build on can change its rules overnight. An email list cannot be deprioritised by an algorithm. This layer costs nothing and is the only asset you keep.

Threshold: none. Do this first.

L2

Affiliate revenue — from roughly a thousand engaged viewers

Recommend the tools you genuinely use in the pipeline itself. Software affiliate programs in the AI and business categories often pay recurring commissions rather than one-off, which compounds. Disclose clearly, recommend only what you actually run, and never let the commission choose the recommendation.

Threshold: a small but engaged audience

L3

Your own product — once you know what they ask for

Your comments and replies will tell you exactly what to build. A template pack, a short course, a tool. Margin is near total and you control the price. Use Prompt 34 to mine your own comment section for the exact language and the unmet need, then Prompt 17 to shape the offer.

Threshold: enough comments to see a pattern

L4

Platform ad revenue — once thresholds are met

Real money, but slow to arrive and entirely outside your control. Treat it as a bonus layer on top of the three you own, never as the plan. Earnings per thousand views vary by an order of magnitude across niches, which is why niche selection (Prompt 11) matters more than upload volume.

Threshold: platform-specific eligibility

L5

Sponsorship and service work — the highest ceiling

Once the channel demonstrates you can build this pipeline, two doors open: brands paying for placement, and businesses paying you to build the same system for them. The second is usually worth more and arrives sooner. Your channel is the portfolio.

Threshold: a visible, consistent body of work



THE PROMPT ARCHITECTURE

The six layers underneath every prompt in this manual. Learn them and you stop needing anyone else's prompt pack, including this one.

Every prompt in this guide is built on the same six layers. Learn the architecture and you stop needing anyone else's prompts — you can write a professional one for any situation in about four minutes.

01

ROLE — assign expertise and a bias

Not "you are a helpful assistant." Give a specific practitioner with a point of view and a thing they are hostile to. "You are a fantasy analyst hostile to camp hype" produces different output from "you are a fantasy expert," because the bias tells the model what to discard. The bias is the useful half.

Weak: "You are a marketing expert." Strong: "You are a copywriter who has watched twenty launches fail and can name the reason each one did."

02

CONTEXT — give it the situation, in variables

Wrap every input in [BRACKETS] so the prompt is reusable. Include the constraints that would change the answer: budget, time, skill level, what has already been tried. Most bad output is caused by missing context, not by a bad model.

The test: could a competent human do this task with only what you supplied? If not, add more.

03

FRAMEWORK — tell it how to think, not just what to produce

This is the layer almost everyone skips, and it is the one that separates a professional prompt from a request. Name the criteria, the order of reasoning, or the dimensions to score against. Without it you get the model's average answer instead of a reasoned one.

Example: "Score each candidate 1-10 on these six signals and show your reasoning before ranking."

04

OUTPUT CONTRACT — specify the shape exactly

Numbered sections. Named fields. Stated lengths. If something downstream will parse it, demand strict JSON and forbid prose around it. Ambiguous output format is the most common reason a prompt has to be run three times.

State what you want, in what order, at what length, in what form.

05

CONSTRAINTS — forbid the failure modes in advance

Name the specific bad output you have seen before. "Never say modern or clean." "Do not list a player without naming the metric." "If the evidence is thin, say so rather than pattern-matching." Constraints do more work than instructions.

Every constraint should correspond to a real failure you have already experienced.

06

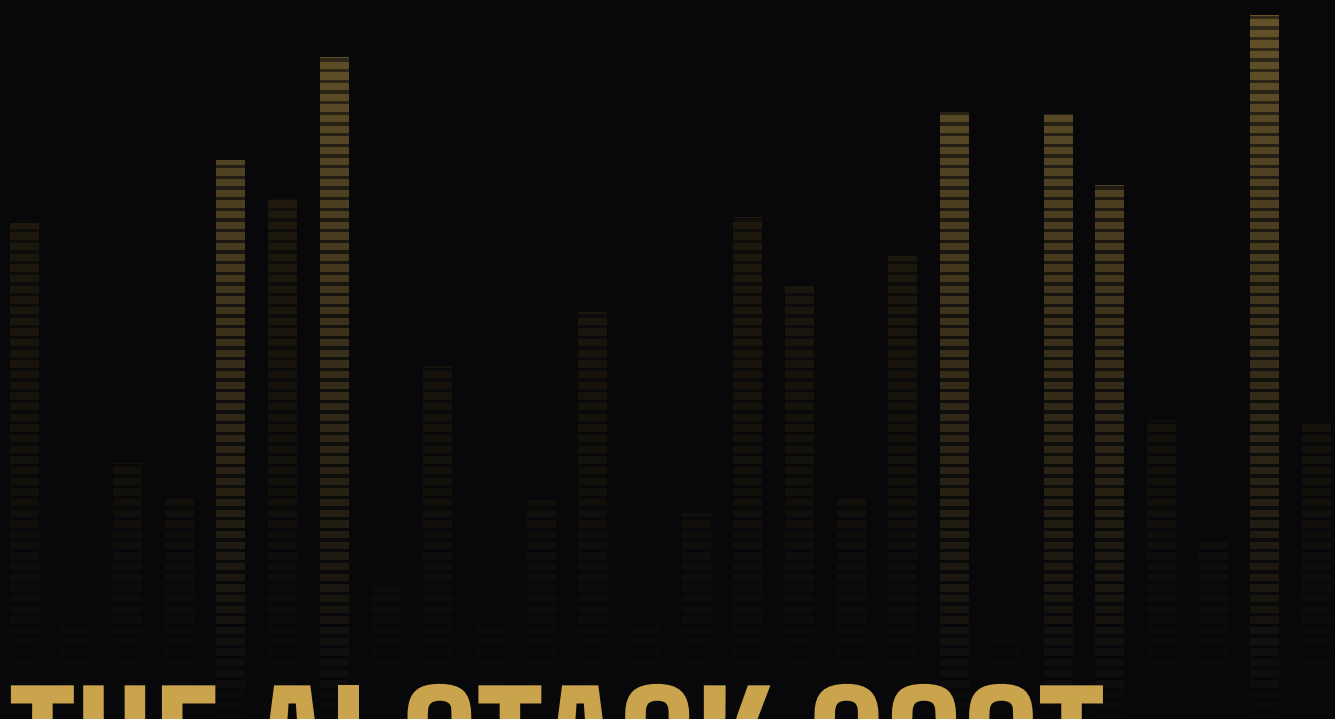
ESCAPE HATCH — tell it what to do when it does not know

Give the model permission to say the request is underspecified, to ask clarifying questions first, or to flag low confidence. Without this it will confidently invent. This one line prevents most fabrication.

Example: "If you are guessing because you cannot see X, say so and ask for X."

THE FOUR-MINUTE METHOD

Write the role with its bias. List every variable in brackets. Name the framework it should reason through. Number the output sections. Add three constraints from past failures. Add the escape hatch. Done — you have just written something better than most paid prompt packs.



THE AI STACK COST LADDER

What to pay for, what to refuse, and the exact signal that says it is time to climb the next rung.

The fastest way to stall is to assemble a \$300 monthly stack before earning a dollar. Spend at each rung only when the rung below is producing. This is the ladder, with the trigger for each step.

RUNG	MONTHLY	WHAT YOU PAY FOR	THE TRIGGER TO CLIMB
0 — Proving	\$0	Free tiers of Claude, ChatGPT, Perplexity. n8n free cloud. Canva free. CapCut. A free Netlify or Lovable page.	You have shipped something and someone responded to it
1 — Building	\$20–40	One AI subscription — whichever you use daily. A domain. Nothing else.	You are hitting usage limits mid-task, more than once a week
2 — Publishing	\$50–80	Second AI subscription. Email platform on a paid tier so automations work. Voice tool if you publish audio.	You have an audience receiving something on a schedule
3 — Selling	\$90–150	Payments live. Scheduling tool. Self-hosted n8n on a small VPS. Perplexity Pro for research volume.	Revenue is arriving and the tools are the bottleneck
4 — Scaling	\$200–400	Generation credits at volume. Analytics. A contractor for the step you hate most.	Your own hours, not your tools, are the constraint

The four rules

- Never pay for a tool to solve a problem you have not yet actually hit. Anticipated bottlenecks are almost always the wrong ones.
- Cancel anything unopened for thirty days. Audit on the first of every month, without sentiment.
- Pay annually only after six months of continuous use. The discount is real; the lock-in on a tool you abandon is more expensive.
- The cheapest upgrade is almost always a better prompt, not a better plan. Exhaust that first.

THE ONE EXCEPTION

Pay for your email platform early, even at rung 1. It is the only asset in the stack you own outright, and free tiers usually disable exactly the automation features that make it worth having.



THE FIRST-DOLLAR TREE

Four income models, and an honest way to find which one fits your actual situation right now rather than your ambition.

There is no universally correct income model. There is a correct one for your situation right now. Answer honestly and the branch you land on is where you should start — not where you should stay forever.

Start here: do you need money within 60 days?

YES — and I have a skill someone would pay for → SERVICES

The fastest first dollar in this entire guide. You are not building an audience, you are solving one business's problem for money. Run Prompt 21 to find the pain, Prompt 22 to reach them, Prompt 23 to run the call, Prompt 24 to close it, and Prompt 25 to make it recurring. Realistic first revenue: two to four weeks. Ceiling: limited by your hours until you productize, which is a good problem to have later.

YES — but I have no sellable skill yet → PRODUCTIZED DELIVERY

Pick one narrow, repeatable output that AI makes you competent at overnight: automation builds, listing optimization, research briefs, content systems. Learn it by building it once for free for someone real, then charge. Prompt 15 and Prompt 21 together will get you there. Realistic first revenue: four to eight weeks.

NO — I can invest 90 days before earning → OWNED MEDIA

Build the asset that compounds. Faceless channel, newsletter, or search-driven content. Slower to first dollar, but it keeps paying and it becomes the distribution for everything you sell later. Prompts 11 through 16 are the whole build. Realistic first revenue: 60 to 120 days. Ceiling: very high.

NO — and I can actually build things → SOFTWARE

A micro-SaaS or paid tool serving a niche you understand. Highest ceiling, highest failure rate, and the only model where the thing you build works while you sleep from day one. Prompts 1 through 6 are the full path. Validate with Prompt 3 before writing a line — most of these die from building the wrong thing, not from building it badly.

THE SEQUENCING THAT ACTUALLY WORKS

Services fund media. Media distributes products. Products become software. Most durable one-person businesses were built in that order, not by picking the highest-ceiling model first and starving on the way there.



THE OPERATING CADENCE

The weekly shape that protects compounding work from urgent work. This is the difference between busy and paid.

A one-person AI business fails from scatter, not from lack of effort. This is the operating cadence — a fixed weekly shape that protects the work that compounds from the work that merely feels urgent.

DAY	BLOCK	THE ONE JOB	NEVER DO THIS DAY
Monday	Build — 3 hrs	The single asset that did not exist last week	Check analytics
Tuesday	Build — 3 hrs	Finish Monday's asset. No new starts.	Start something new
Wednesday	Distribute — 2 hrs	Ship it and put it where people are	Polish
Thursday	Revenue — 2 hrs	Outreach, follow-ups, or the sales conversation	Create content
Friday	Review — 45 min	The five questions below. Then stop.	Add to next week

The Friday five

- What did I ship this week that a stranger could see and use?
- What did I spend the most hours on, and did it produce revenue or the feeling of revenue?
- What did I avoid, and what is the real reason I avoided it?
- What is the single most important thing next week, if I only get one thing done?
- What am I going to stop doing entirely, starting Monday?

The three rules that hold it together

- One project at a time until it produces revenue or is formally abandoned. Two half-finished projects have less value than one finished one — not half the value, less.
- Ship before it is ready, every single week. The feedback from a live thing is worth more than another day of private polish, and the polish rarely survives contact with real users anyway.
- Protect Thursday. Every failing one-person business has a full content calendar and an empty pipeline. Revenue activity is the first thing to get skipped and the last thing that should be.

THE COMPOUNDING TEST

Before any task, ask: will this still be producing value in six months? Content ranks, systems run, assets sell, relationships return. Everything else is maintenance — necessary, but it should never occupy your best hours.



YOU HAVE THE SYSTEM. NOW SHIP SOMETHING.

The full AI Income Systems Lab goes further: complete end-to-end builds where every prompt, workflow and asset is assembled into a running system — and the monthly prompt packs that keep the stack current as these tools change.

- 25 ranked AI income blueprints, scored on profit, ease, automation and demand
- The Workflow Rescue Co. build — a complete AI automation agency, end to end
- AI Front Desk · AI Lead Forge · Faceless Content Factory · Newsletter Engine
- Monthly prompt packs as the models and tools change underneath you
- The All-Access membership: live builds, office hours, and the community

AI-INCOME-SYSTEMS.COM